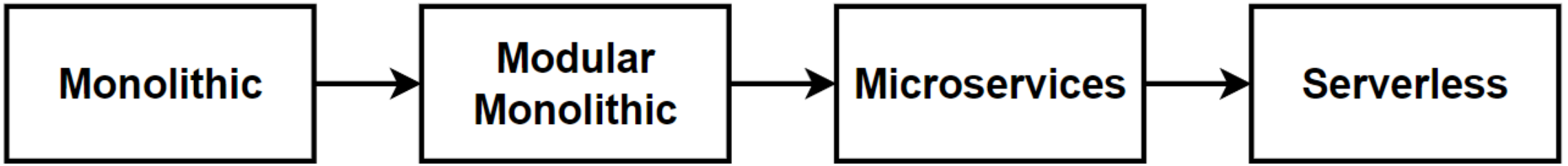
Design Microservices Architecture with Patterns & Best Practices

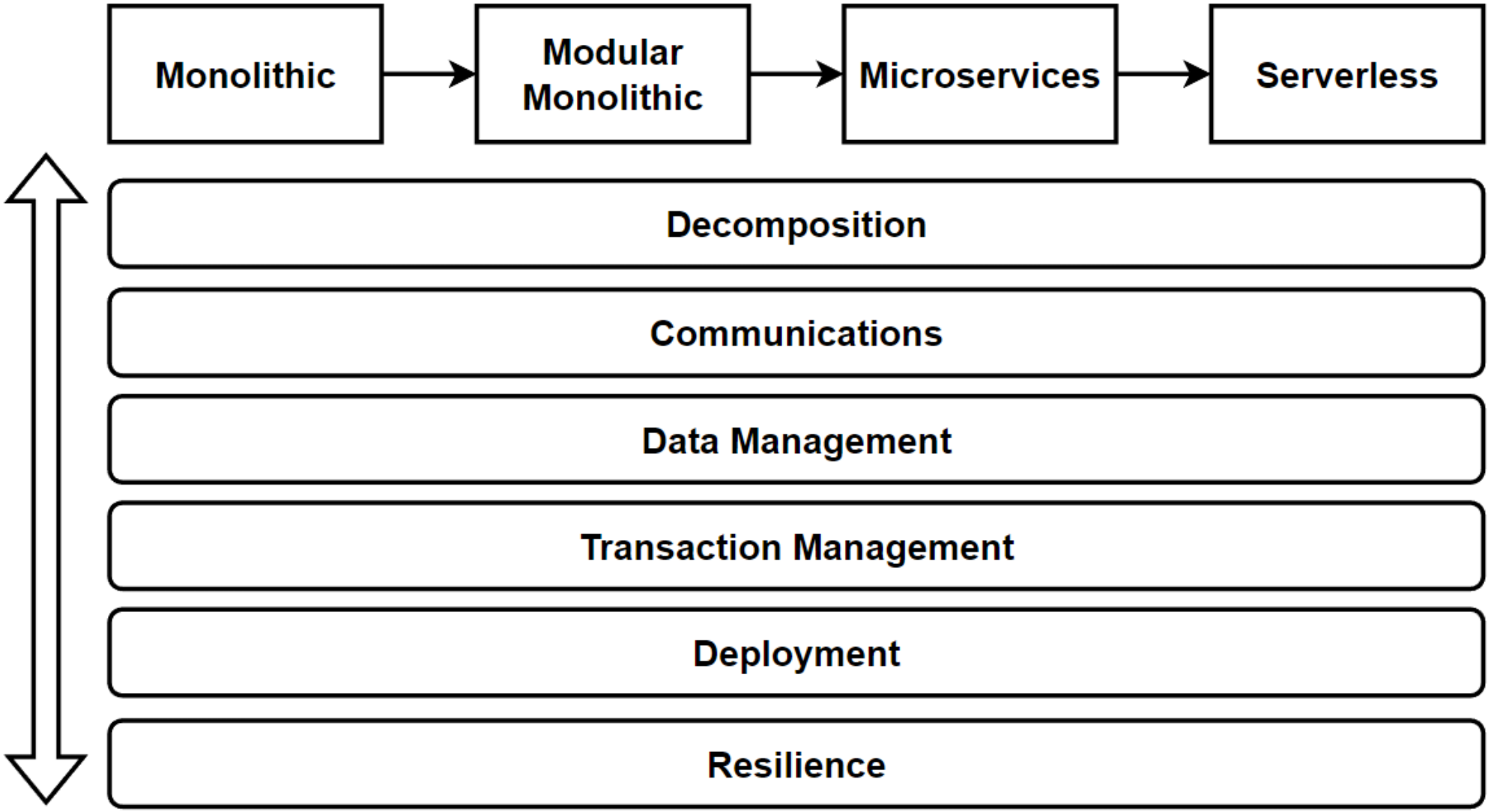
A step-by-step process for software system design and evolve from monolithic to microservices following the patterns & principles.

Refactor architectures with different aspects of microservices pillars.

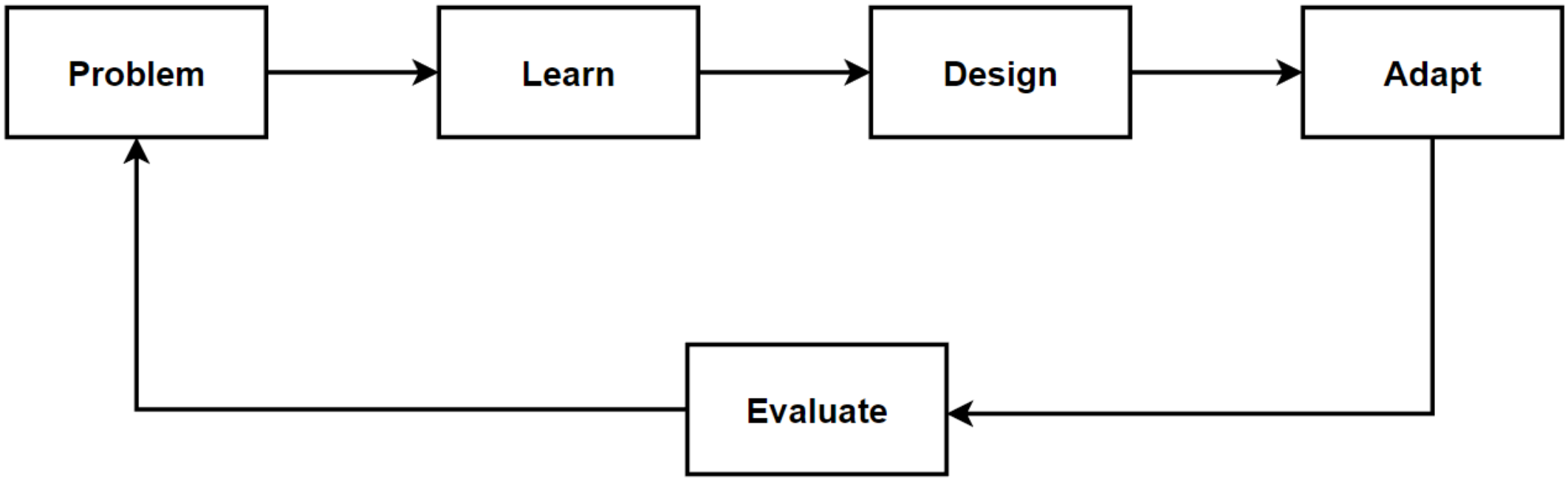
Architecture Design Journey



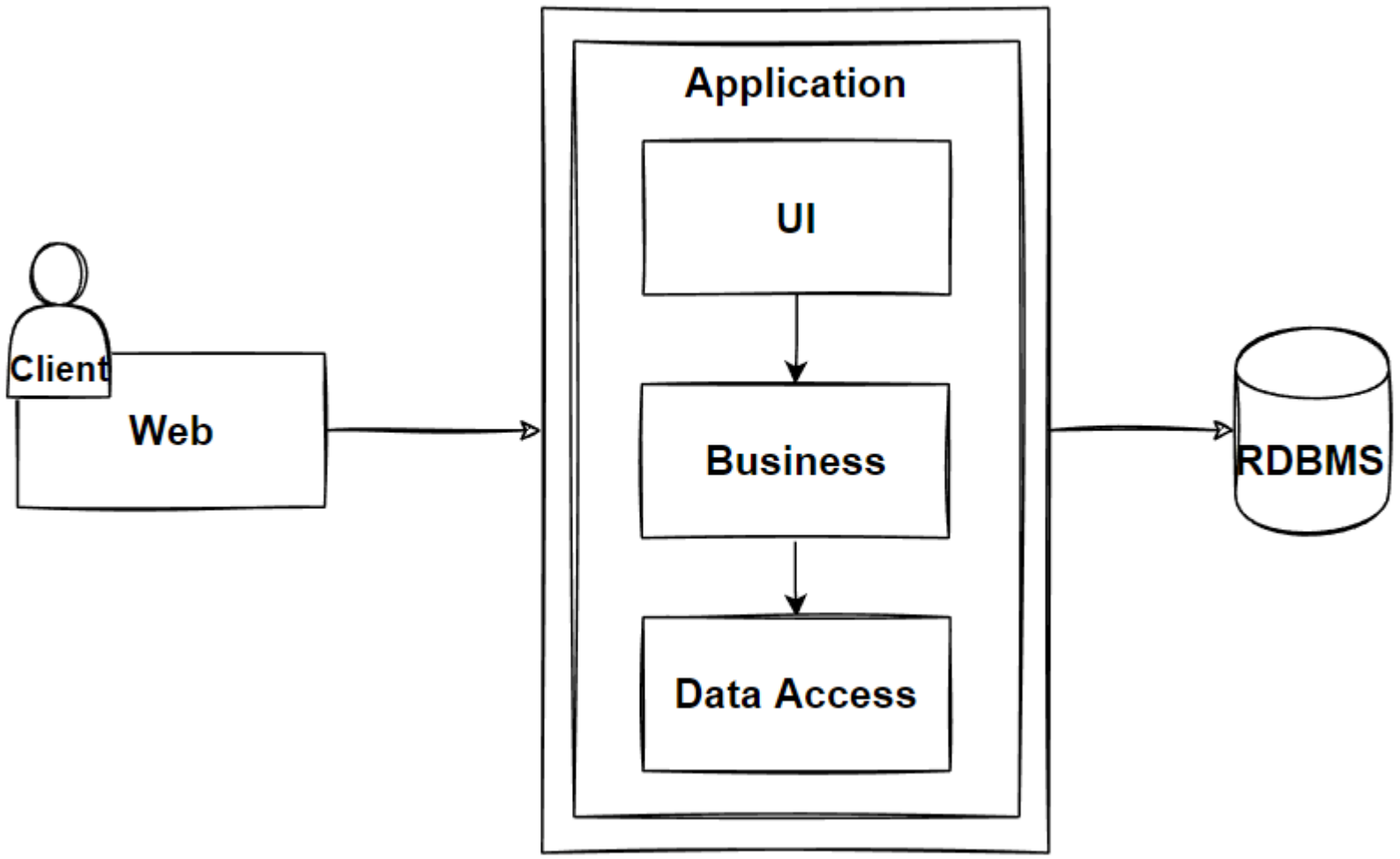
Architecture Design – Vertical Considerations



Way of Learning – The Course Flow



Monolithic Architecture



Modular Monolithic Architecture Patterns

Architectures

- Monolithic Architecture
- Layered Architecture
- Clean Architecture
- **Modular Monolithic Architecture**

Patterns&Principles

- KISS
- YAGNI
- Separation of Concerns (SoC)
- SOLID
- The Dependency Rule
- Horizontal Scaling
- Load Balancer
- **Monolithic-First Strategy**

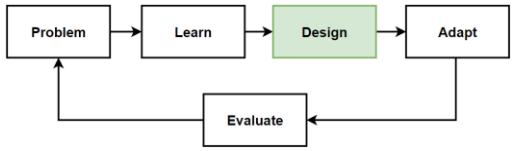
Non-FR

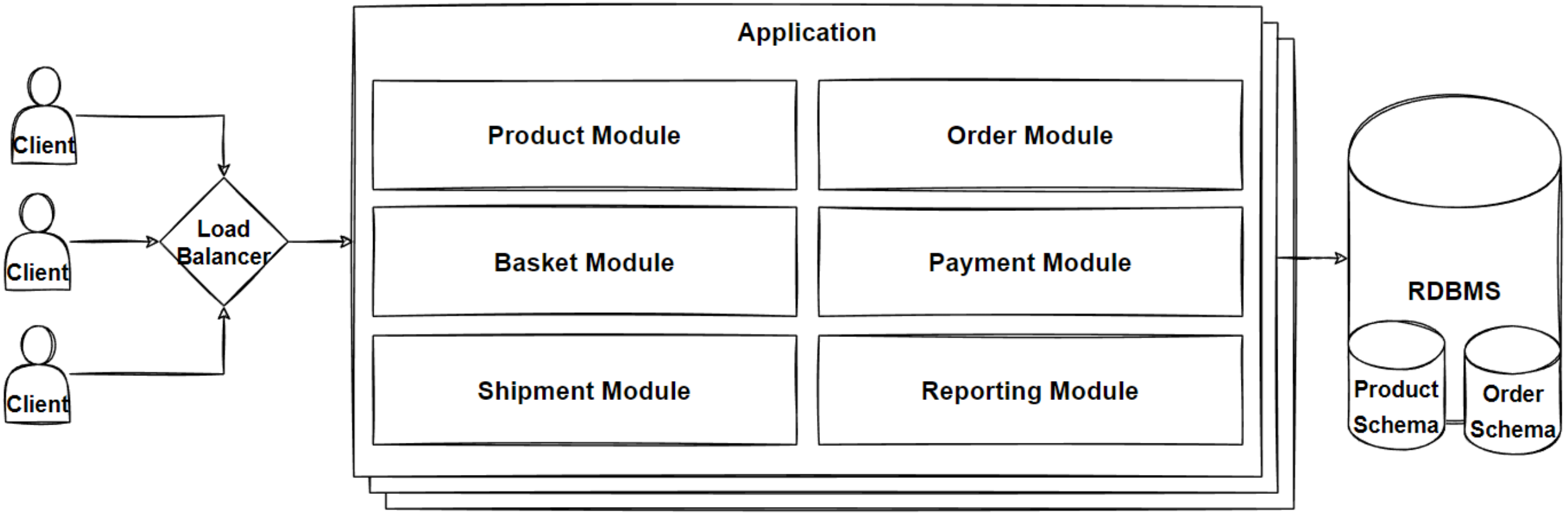
- Availability
- High number of Concurrent User
- Maintainability
- Flexibility
- Testable
- Scalability
- **Reliability**
- **Re-usability**

FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Design: Modular Monolithic Architecture





Microservices Architecture Patterns

Architectures

- **Microservices Architecture**

Patterns&Principles

- **The Database-per-Service Pattern**
- **Polygot Persistence**

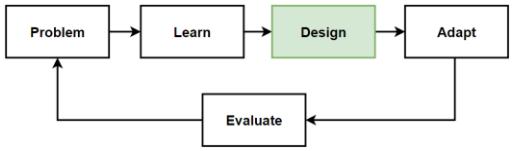
Non-FR

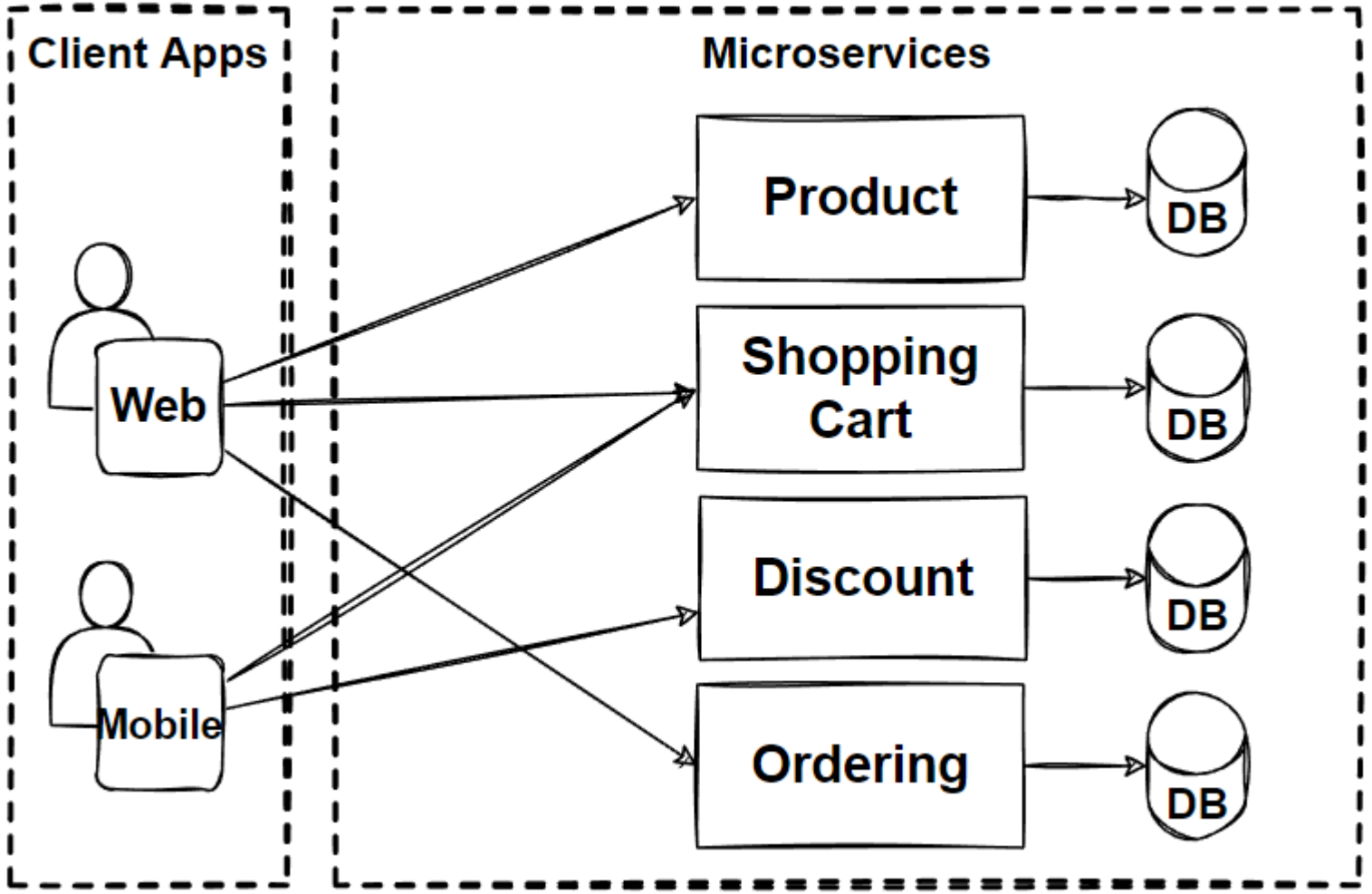
- **High Scalability**
- **High Availability**
- **Millions of Concurrent User**
- **Independent Deployable**
- **Technology agnostic**
- **Data isolation**
- **Resilience and Fault isolation**

FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Design: Microservices Architecture





Microservices Communications

Architectures

- **Microservices Architecture**

Patterns&Principles

- The Database-per-Service Pattern
- Polygot Persistence
- Decompose services by scalability
- The Scale Cube
- Microservices Decomposition Pattern
- **Microservices Communications**
  - **HTTP Based RESTful API design**
  - **GraphQL API design**
  - **gRPC API Design**

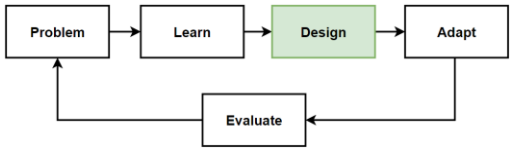
Non-FR

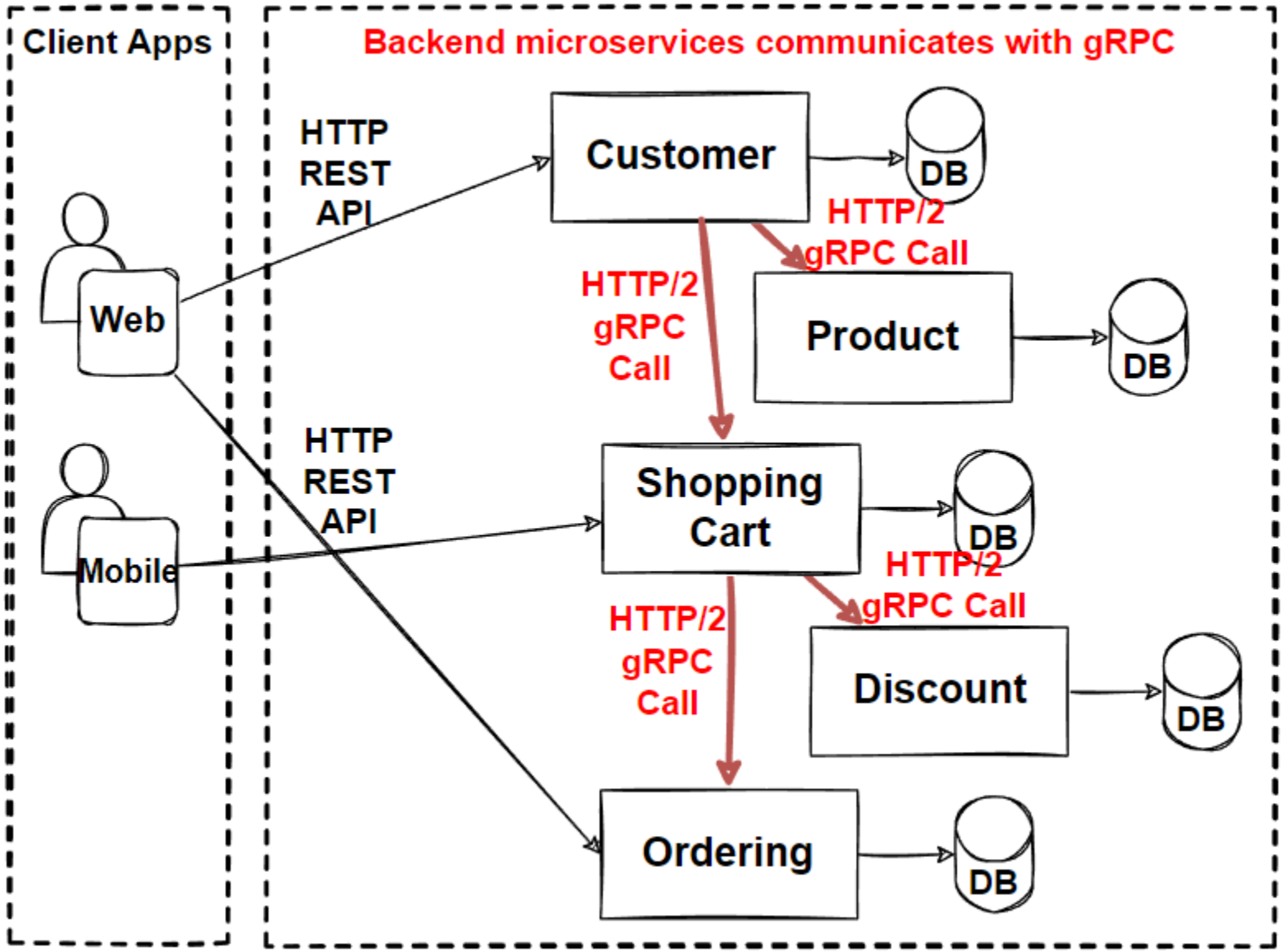
- High Scalability
- High Availability
- Millions of Concurrent User
- Independent Deployable
- Technology agnostic
- Data isolation
- Resilience and Fault isolation

FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Design: Microservices Architecture with gRPC APIs





Microservices Communications Patterns

Architectures

- **Microservices Architecture**

Patterns&Principles

- The Database-per-Service Pattern
- Polygot Persistence
- Decompose services by scalability
- The Scale Cube
- Microservices Decomposition Pattern
- **Microservices Communications Patterns**

Microservices Communications

- HTTP Based RESTful API
- GraphQL API
- gRPC API
- WebSocket API
- Gateway Routing Pattern
- Gateway Aggregation Pattern
- Gateway Offloading Pattern
- **API Gateway Pattern**
- **Backends for Frontends Pattern-BFF**

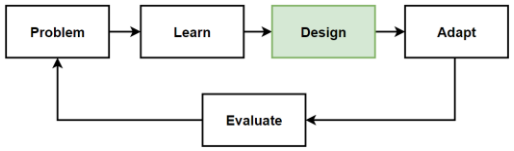
Non-FR

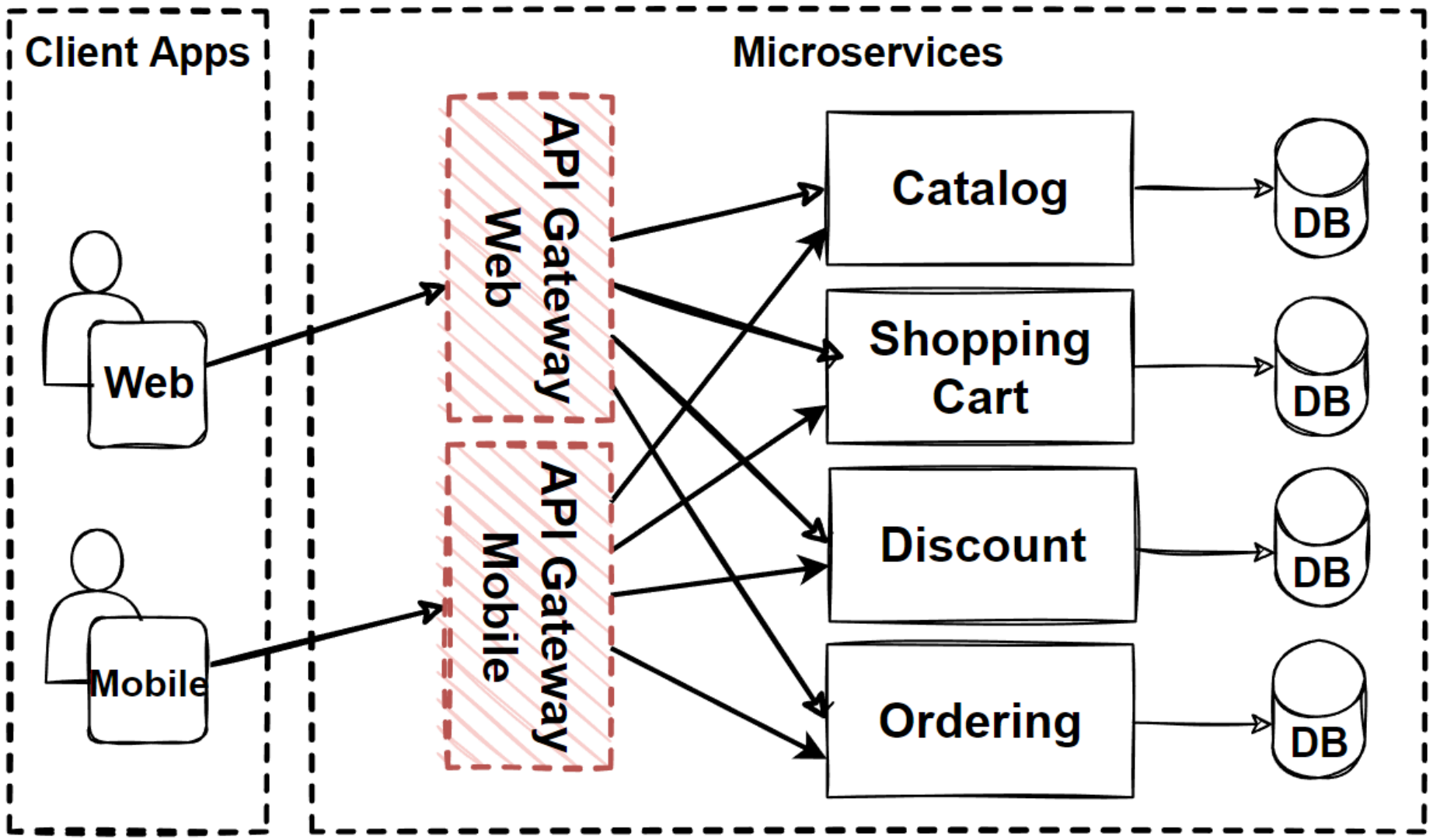
- High Scalability
- High Availability
- Millions of Concurrent User
- Independent Deployable
- Technology agnostic
- Data isolation
- Resilience and Fault isolation

FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Design:Microservices Architecture with BFF





Microservices Async Communications Patterns

Architectures

- **Microservices Architecture**

Patterns&Principles

- The Database-per-Service Pattern
- Polygot Persistence
- Decompose services by scalability
- The Scale Cube
- Microservices Decomposition Pattern
- **Microservices Communications Patterns**

Microservices Communications

- HTTP Based RESTful API
- GraphQL API
- gRPC API
- WebSocket API
- Gateway Routing Pattern
- Gateway Aggregation Pattern
- Gateway Offloading Pattern
- API Gateway Pattern
- Backends for Frontends Pattern-BFF
- Service Aggregator Pattern
- Service Registry/Discovery Pattern

Microservices Async Communications

- **Single-receiver Message-based Communication (one-to-one model)**
- **Multiple-receiver Message-based Communication (one-to-many model-topic)**
- **Dependency Inversion Principles (DIP)**
- **Fan-Out Publish/Subscribe Messaging Pattern**
- **Topic-Queue Chaining & Load Balancing Pattern**

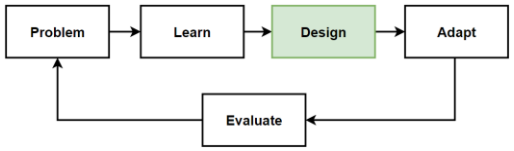
FR

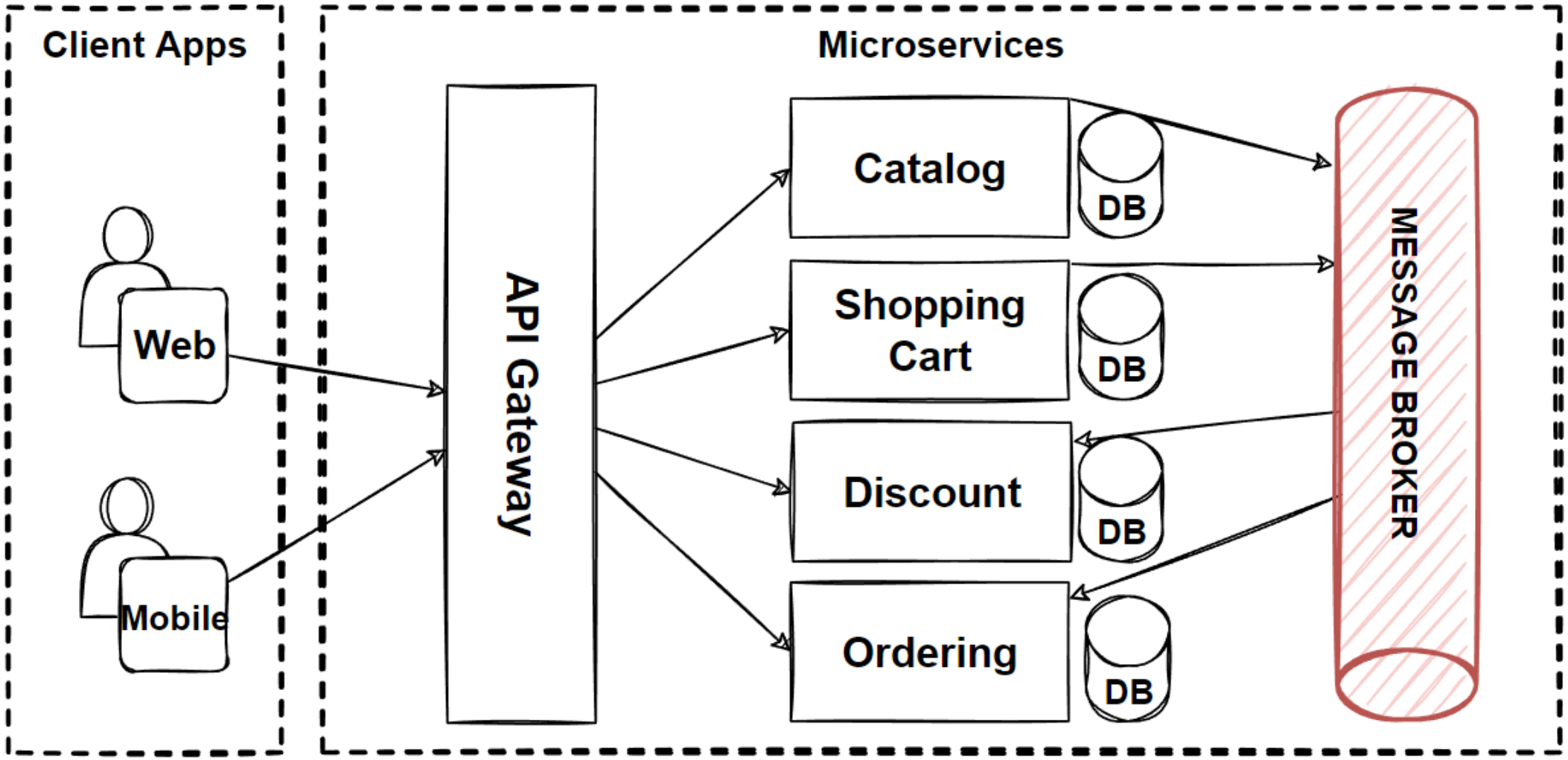
- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an orde
- List my old orders and order items history

Non-FR

- High Scalability
- High Availability
- Millions of Concurrent User
- Independent

Design: Microservices Architecture with Fan-Out Publish/Subscribe Messaging Pattern





Microservices Data Management Patterns

Architectures Patterns&Principles

- **Microservices Architecture**
  - The Database-per-Service Pattern
  - Polygot Persistence
  - Decompose services by scalability
  - The Scale Cube
  - Microservices Decomposition Pattern
  - Microservices Communications Patterns
  - **Microservices Data Management Patterns**

Microservices Data Choosing Database

- The Shared Database Anti-pattern
- Relational and NoSQL Databases
- CAP Theorem–Consistency, Availability, Partition Tolerance
- Data Partitioning: Horizontal, Vertical and Functional Data Partitioning
- Database Sharding Pattern

Microservices Data Commands&Queries

- **Materialized View Pattern**
- **CQRS Design Pattern**
- **Event Sourcing Pattern**
- **Eventual Consistency Principle**

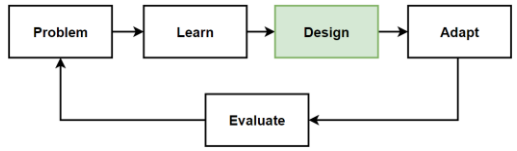
FR

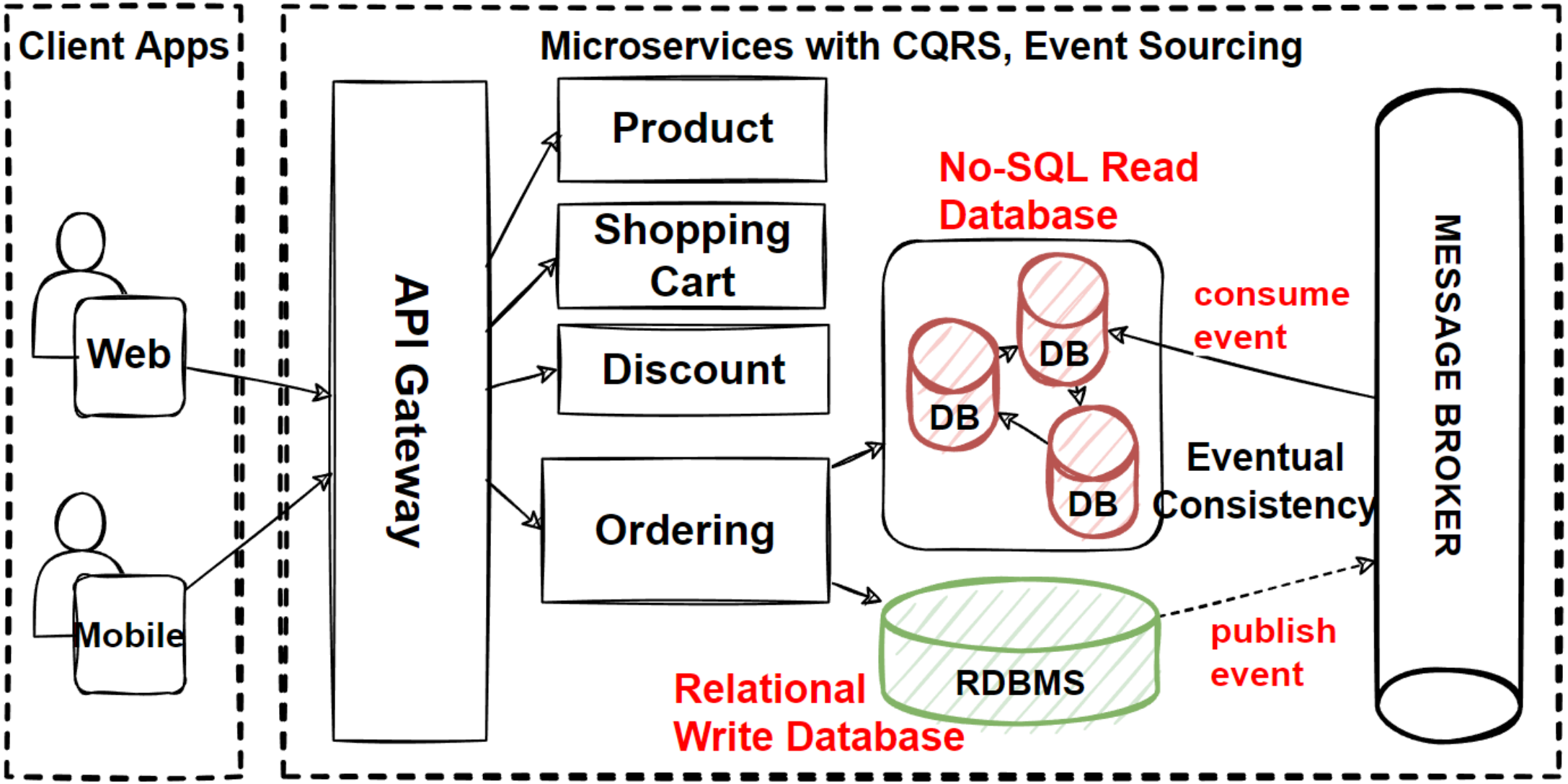
- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an orde
- List my old orders and order items history

Non-FR

- High Scalability
- High Availability
- Millions of Concurrent User
- Independent

Design: Microservices Architecture with CQRS, Event Sourcing, Eventual Consistency





Microservices Distributed Transaction Patterns

Architectures Patterns&Principles

- **Microservices Architecture**
  - The Database-per-Service Pattern
  - Polygot Persistence
  - Decompose services by scalability
  - The Scale Cube
  - Microservices Decomposition Pattern
  - Microservices Communications Patterns
  - Microservices Data Management Patterns
  - **Microservices Distributed Transaction Pattern**

Microservices Data Choosing Database

- The Shared Database Anti-pattern, Relational and NoSQL Databases
- CAP Theorem–Consistency, Availability, Partition Tolerance
- Data Partitioning: Horizontal, Vertical and Functional Data Partitioning
- Database Sharding Pattern

Microservices Data Commands&Queries

- Materialized View Pattern
- CQRS Design Pattern
- Event Sourcing Pattern
- Eventual Consistency

Microservices Distributed Transactions

- **SAGA Pattern**
- **Choreography and Orchestration-based SAGA**
- **Compensating Transaction Pattern**
- **Dual-Write Problem**
- **Transactional Outbox Pattern**
- **CDC - Change Data Capture**

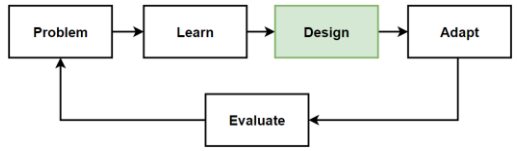
FR

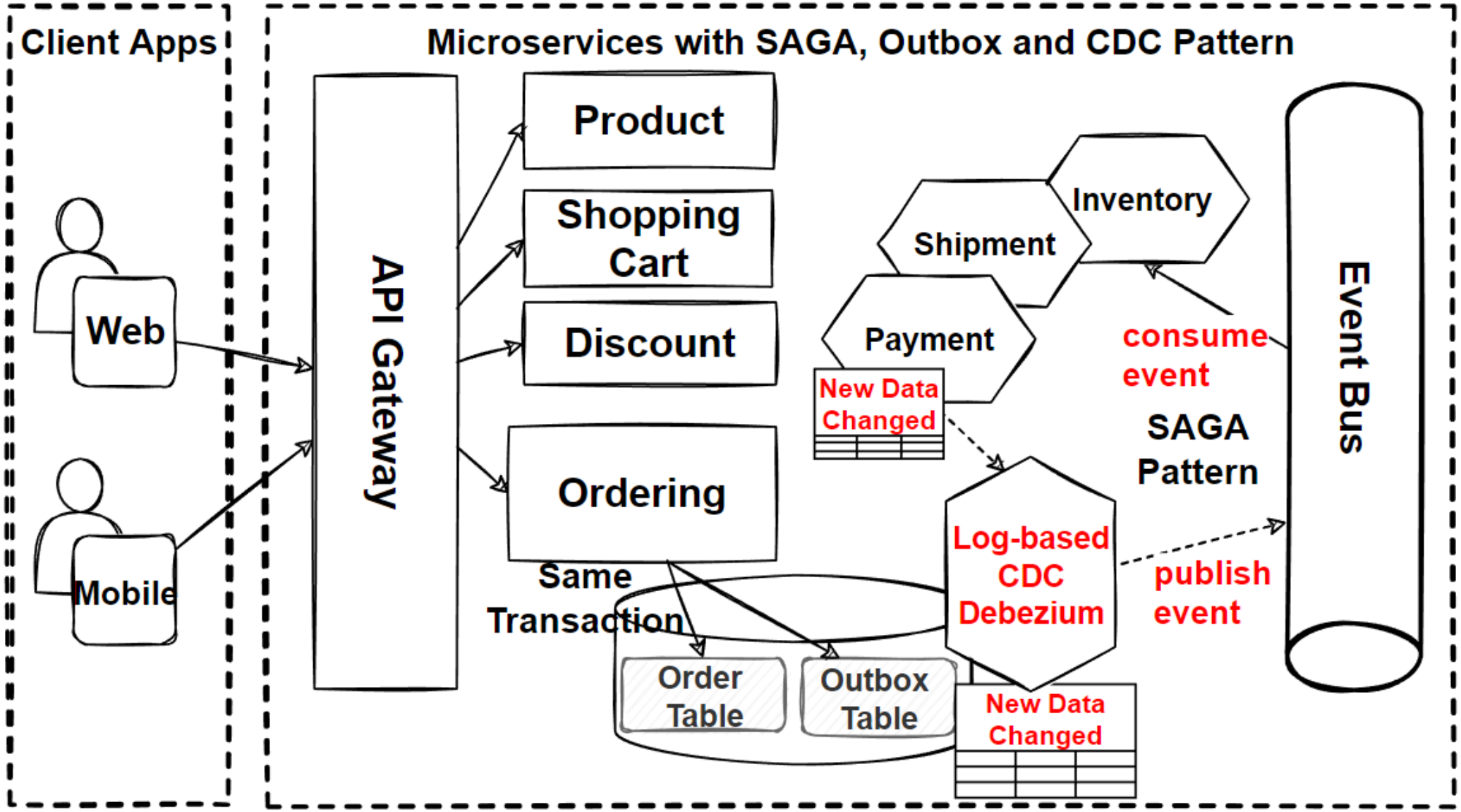
- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an orde
- List my old orders and order items history

Non-FR

- High Scalability
- High Availability
- Millions of Concurrent User
- Independent

Microservices with SAGA, Transactional Outbox and CDC Pattern





Event-Driven Microservices Patterns

Architectures

- Microservices Architecture
- **Event-Driven Microservices Architecture**

Patterns&Principles

- The Database-per-Service Pattern
- Polygot Persistence
- Decompose services by scalability
- The Scale Cube
- Microservices Decomposition Pattern
- Microservices Communications Patterns
- Microservices Data Management Patterns
- **Event-Driven Architecture**

Microservices EDA

- **Asynchronous, Decoupled communication**
- **Event Hubs**
- **Stream-Processing**
- **Real-time processing**
- **High volume events**

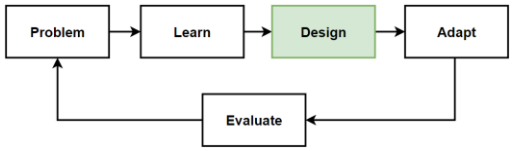
Non-FR

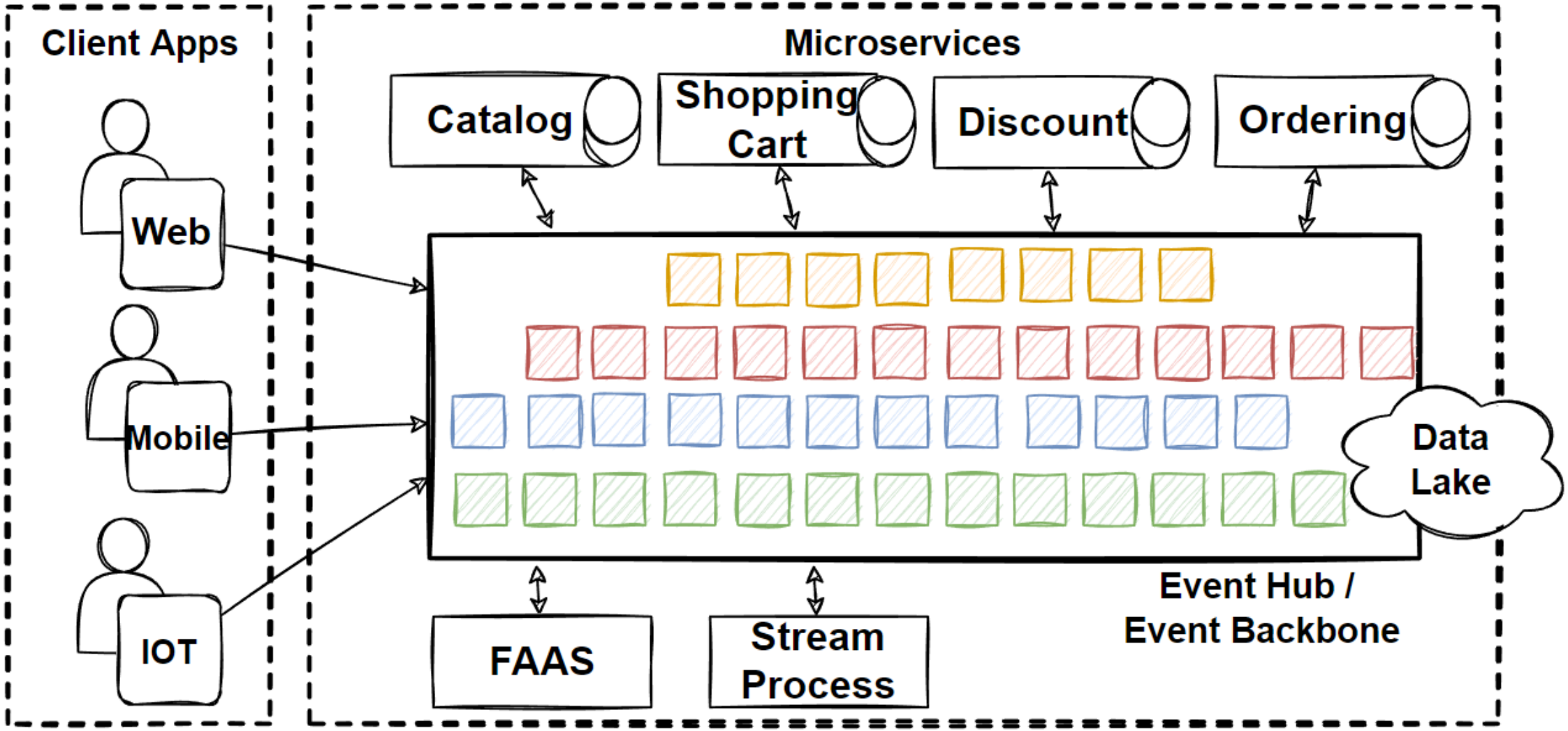
- High Scalability
- High Availability
- Millions of Concurrent User
- Independent Deployable
- Technology agnostic
- Data isolation
- Resilience and Fault isolation

FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Event-Driven Microservices Architecture





Microservices Deployment Patterns

Architectures

- **Microservices Architecture**
- Event-Driven Microservices Architecture

Patterns&Principles

- The Database-per-Service Pattern, Polygot Persistence, Decompose services by scalability, The Scale Cube
- Microservices Decomposition Pattern
- Microservices Communications Patterns
- Microservices Data Management Patterns
- Event-Driven Architecture
- Microservices Distributed Caching
- **Microservices Deployments with Containers and Orchestrators**

Microservices Deployment

- **Docker and Kubernetes Architecture, Helm Charts**
- **Kubernetes Patterns; Sidecar Patterns, Service Mesh Pattern**
- **DevOps and CI/CD Pipelines**
- **Deployment Strategies; Blue-green, Rolling, Canary and A/B Deployment.**
- **Infrastructure as code (IaC)**

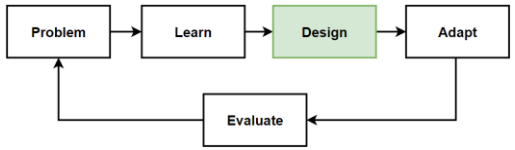
Non-FR

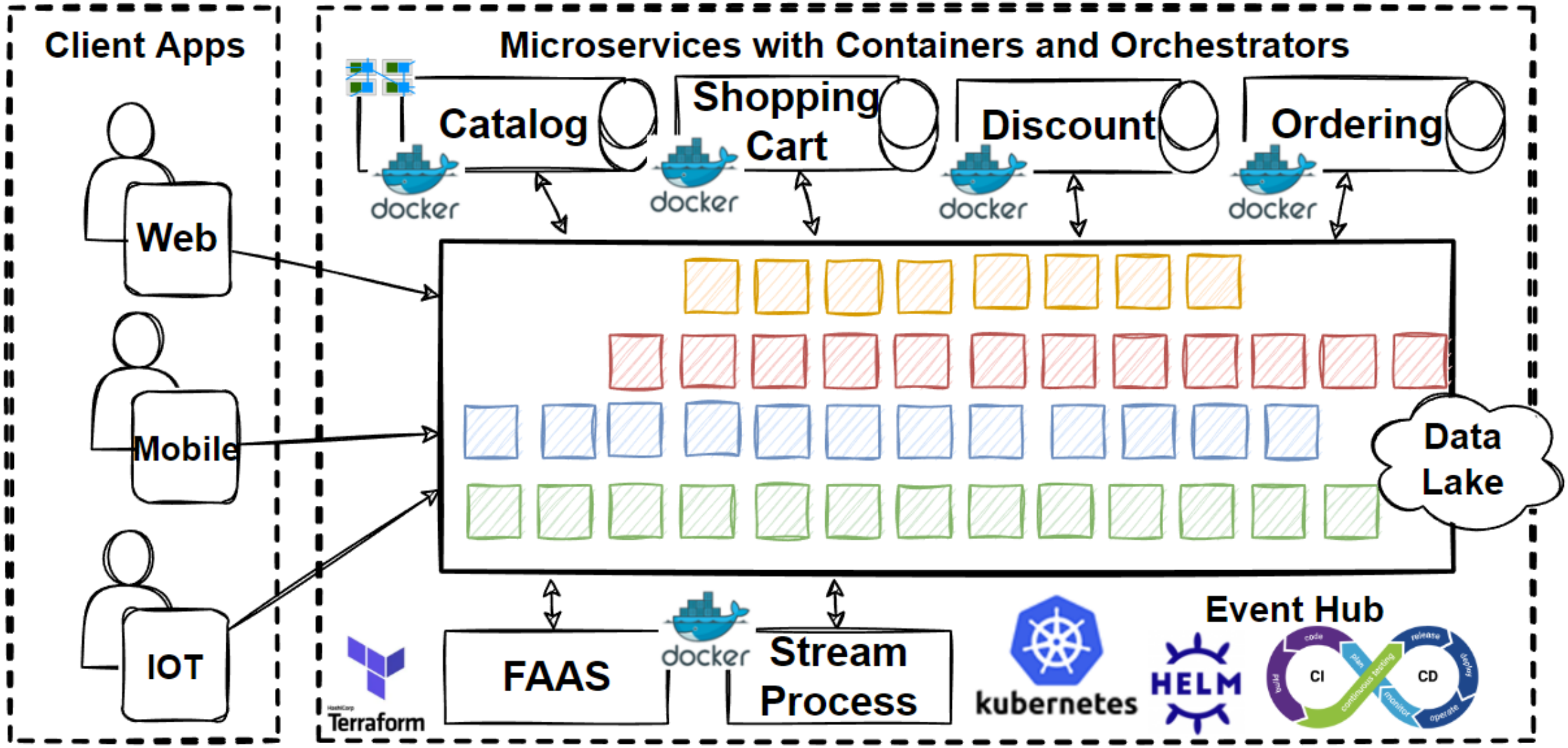
- High Scalability
- High Availability
- Millions of Concurrent User
- Independent Deployable
- Technology agnostic
- Data isolation
- Resilience and Fault isolation

FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Microservices using Containers and Orchestrators





Microservices Resilience Patterns

Architectures

- **Microservices Architecture**
- Event-Driven Microservices Architecture

Patterns&Principles

- The Database-per-Service Pattern, Polygot Persistence, Decompose services by scalability, The Scale Cube
- Microservices Decomposition Pattern
- Microservices Communications Patterns
- Microservices Data Management Patterns
- Event-Driven Architecture
- Microservices Distributed Caching
- Microservices Deployments with Containers and Orchestrators

Microservices Resilience

- **Resilience Patterns; Retry, Circuit-Breaker, Bulkhead, Timeout, Fallback Pattern**
- **Distributed Logging and Distributed Tracing**
- **Elastic Stack; Elasticsearch + Logstash + Kibana**
- **OpenTelemetry using Zipkin**
- **Kubernetes Health Monitoring with tools like Prometheus and Grafana**

Non-FR

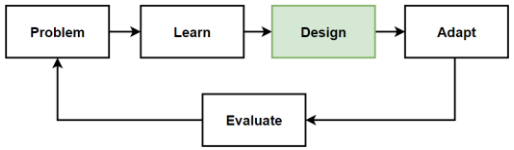
- High Scalability
- High Availability
- Millions of Concurrent User
- Independent Deployable
- Technology agnostic
- Data isolation
- Resilience and Fault isolation

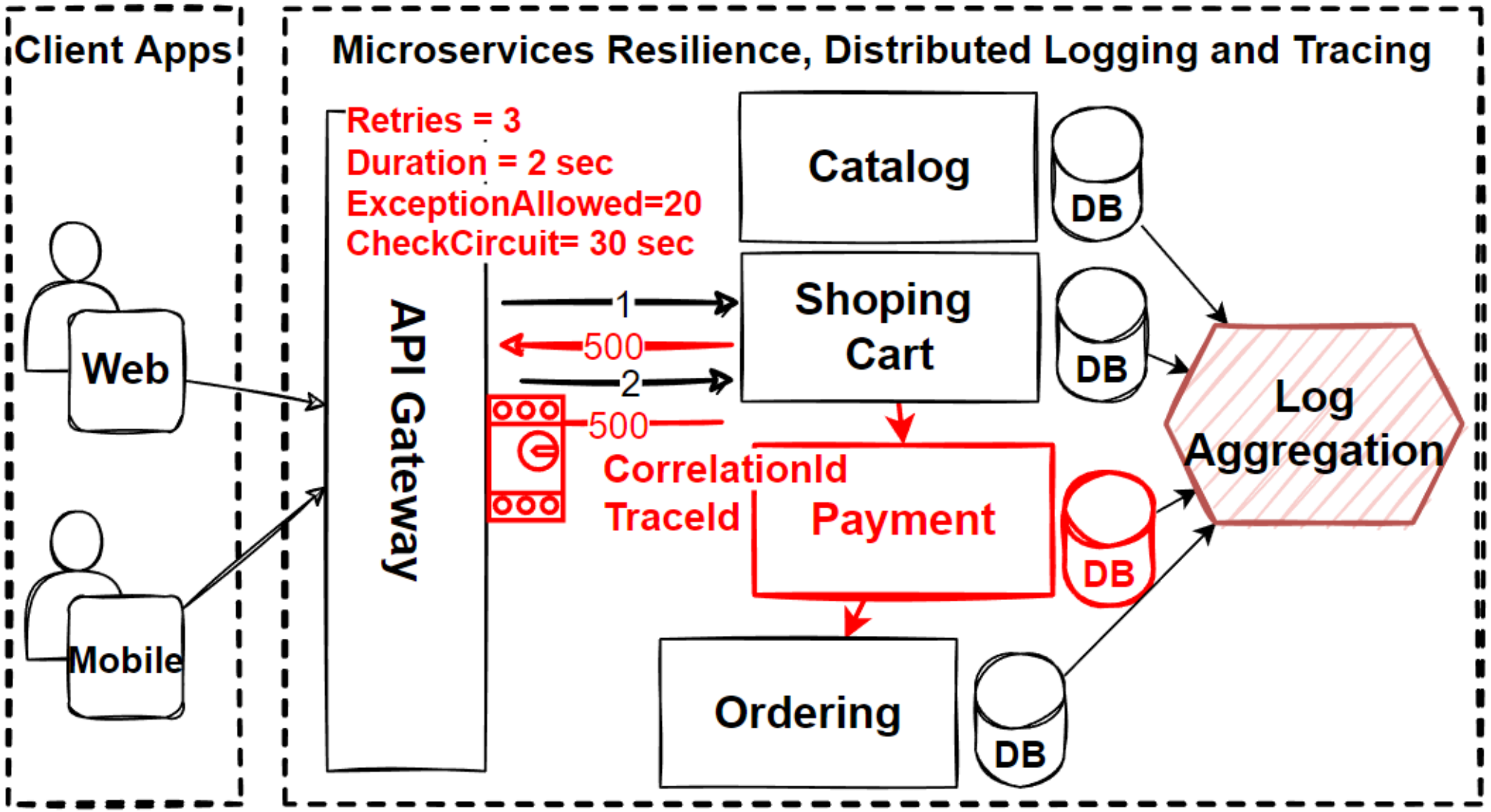
FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

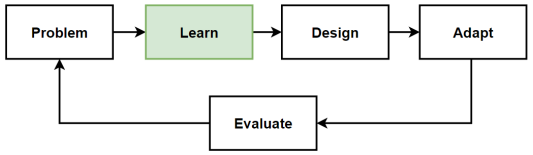
- **Microservices Resilience, Observability and Monitoring**

Microservices Resilience, Observability and Monitoring





Design: Serverless E-Commerce Microservices



AWS Cloud

User

AWS API Gateway

Product

Basket

Order

Product Microservice

AWS Lambda

AWS DynamoDB

Basket Microservice

AWS Lambda

AWS DynamoDB

Ordering Microservice

AWS Lambda

AWS DynamoDB

Checkout Event

AWS SQS Queue

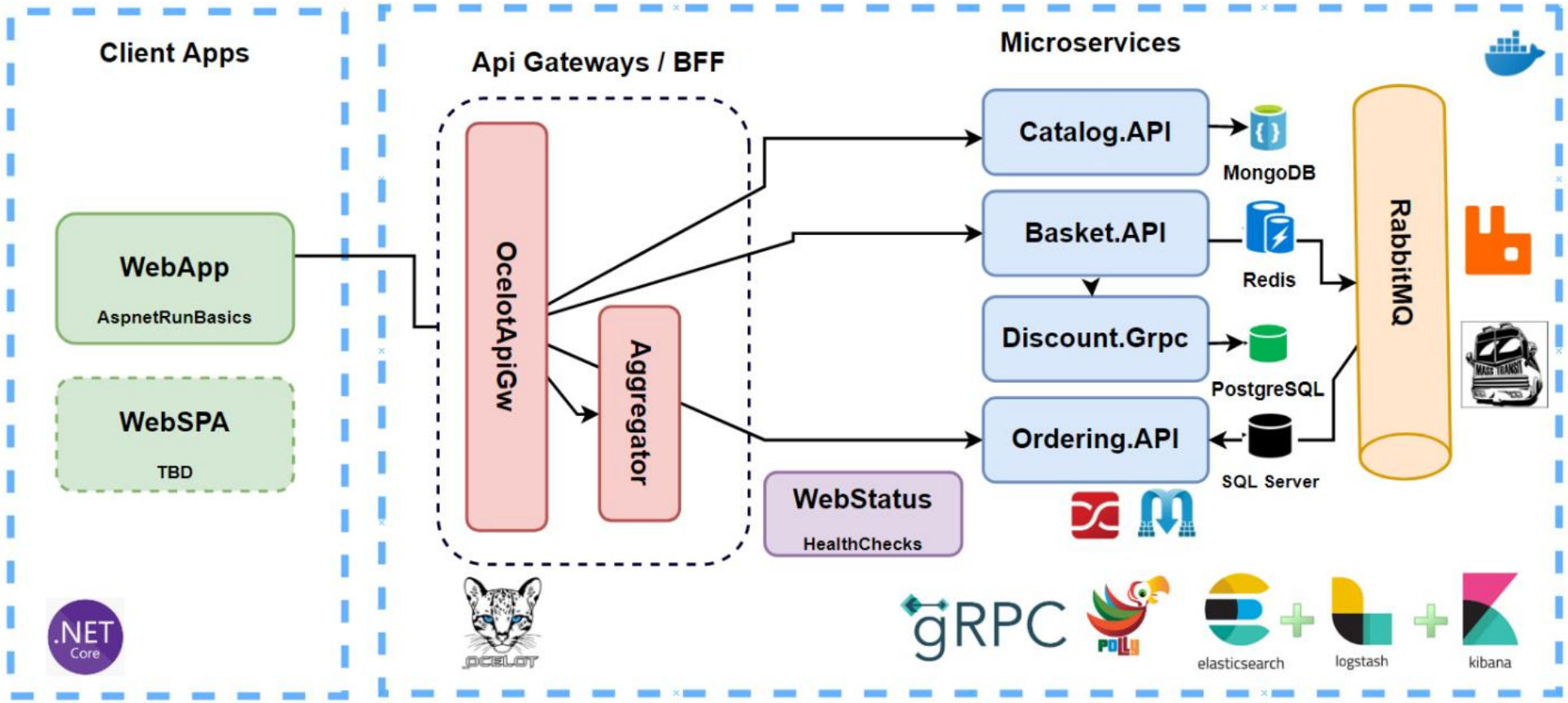
AWS EventBridge Event bus

EventBridge Rules

Stack

CDK CloudFormation

DEMO: Microservices Architecture Code Review



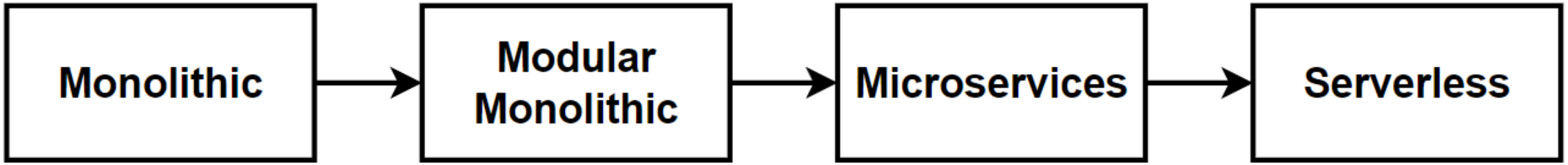
DEMO: Code review of Microservices Architecture .NET Implementation

- https://github.com/aspnetrun/run-aspnetcore-microservices
- https://github1s.com/aspnetrun/run-aspnetcore-microservices

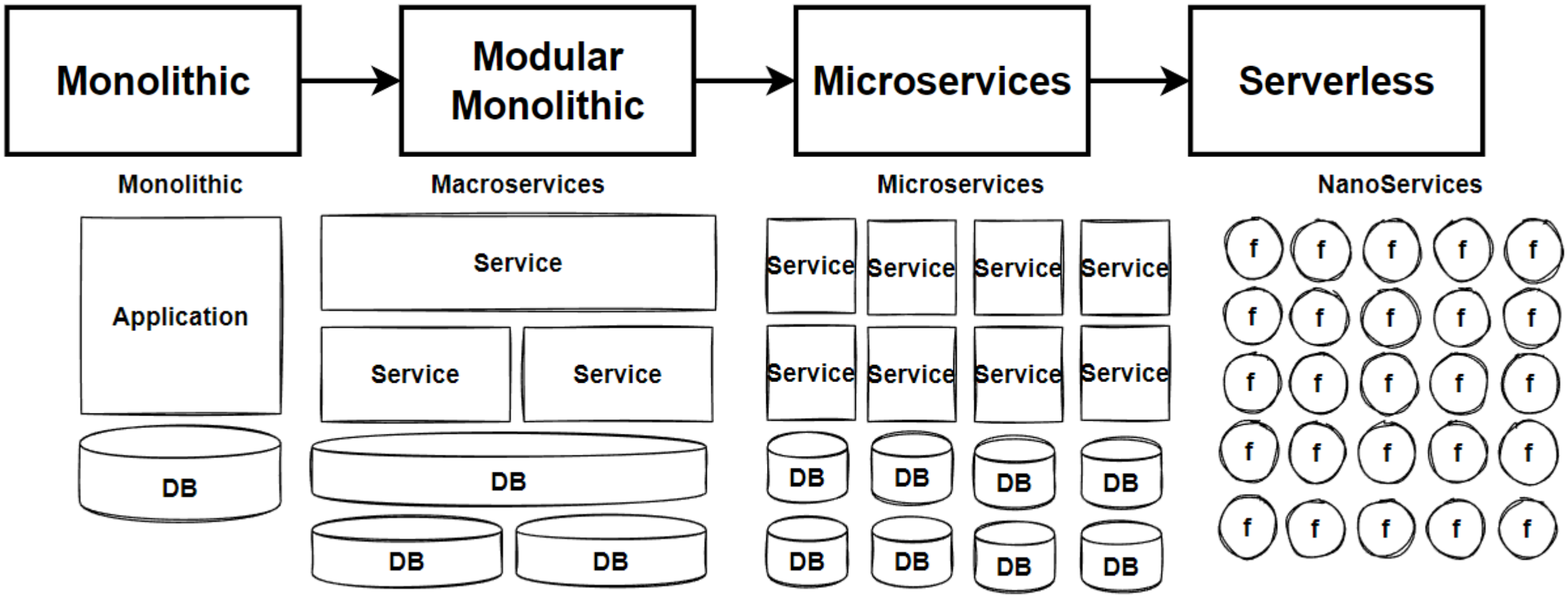
Course Target

- **Hands-on Design** Activities
- **Iterate** Design Architecture from On-Premises to Cloud Serverless
- **Evolves architecture** Monolithic to Event-driven Microservices
- **Refactoring System Design** for handling million of requests
- Apply **best practices** with microservices design patterns and principles
- Examine microservices patterns **with all aspects** like Communications, Data Management, Caching and Deployments
- Prepare for **Software Architecture Interviews**
- Prepare for **System Design Architecture Interview exams**

Architecture Design Journey

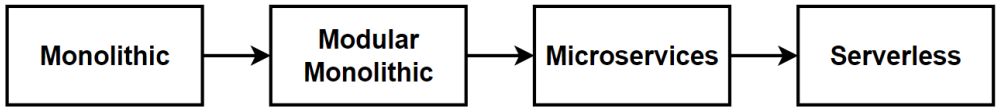


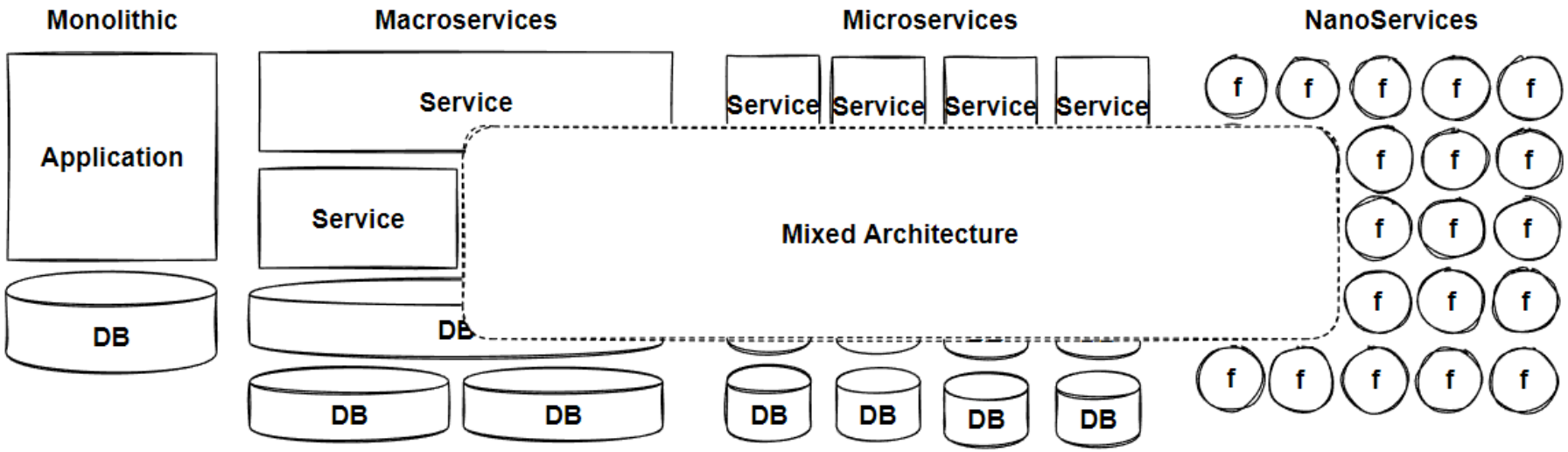
Macroservices to Nanoservices



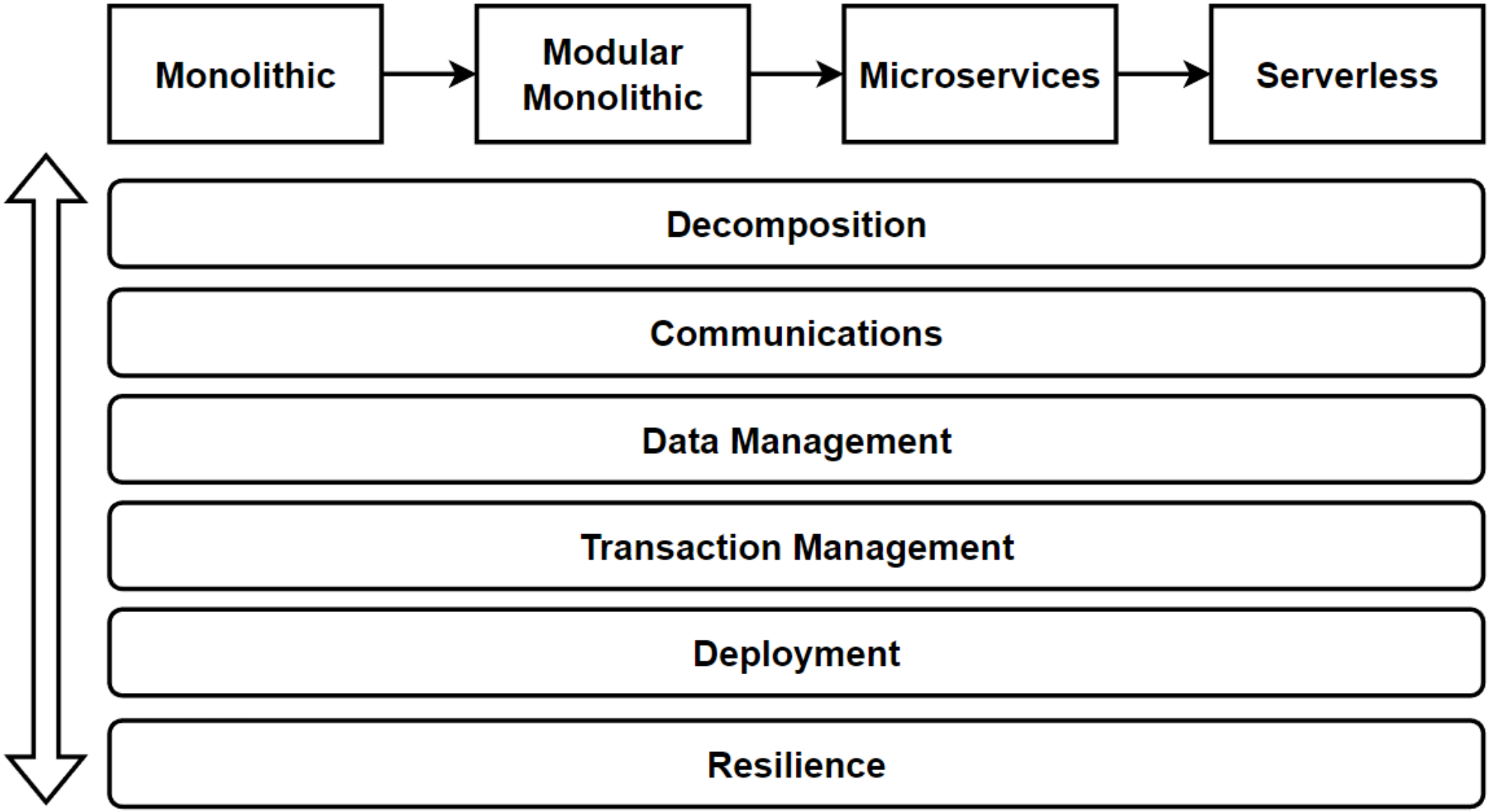
Which architecture approach we should choose ?

- It depends on your project
- Use Mixed Architecture
- Provide transitions between architectures

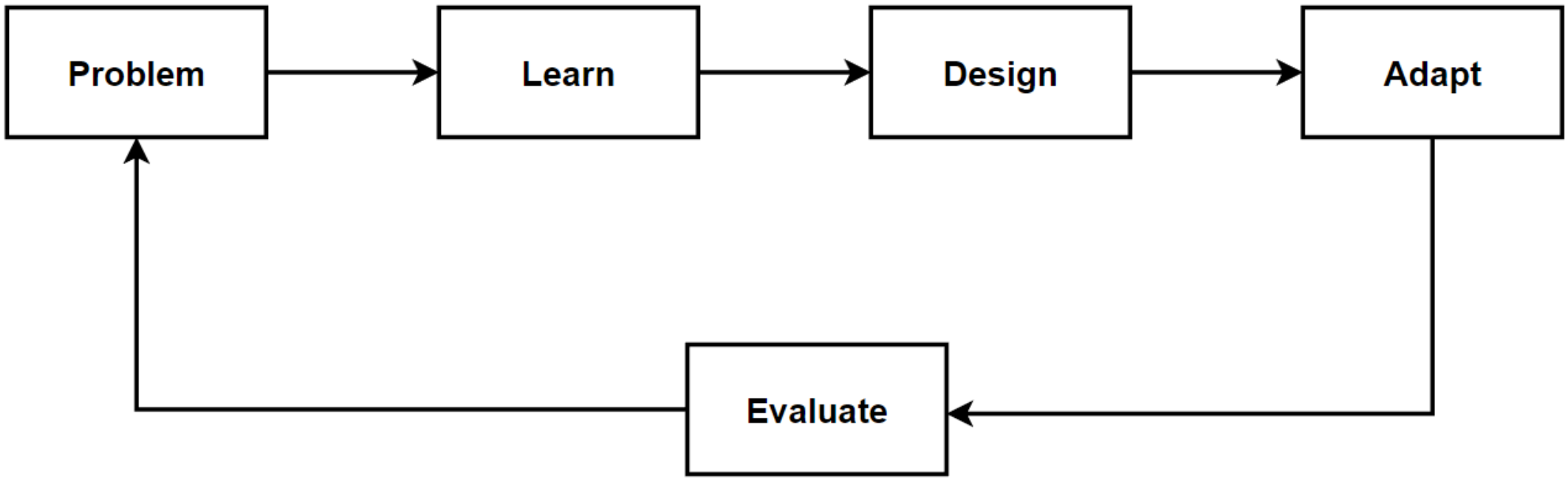




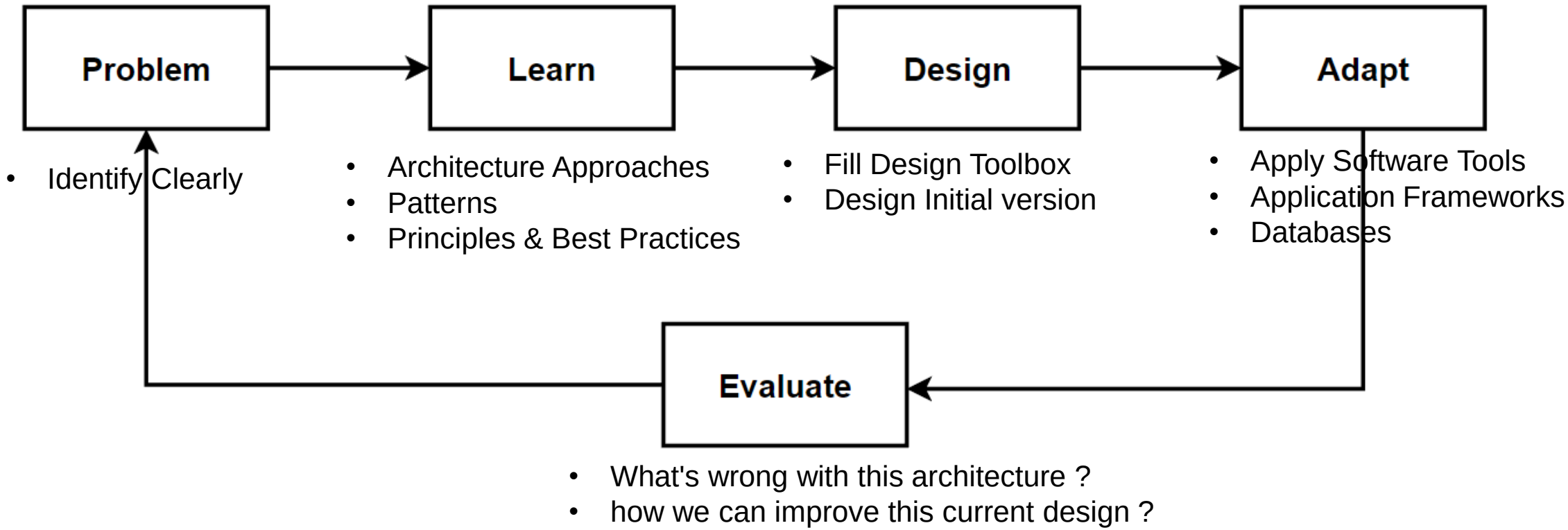
Architecture Design – Vertical Considerations



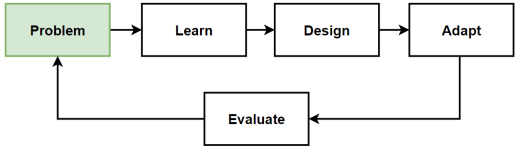
Way of Learning – The Course Flow



Way of Learning – The Course Flow



Problem: Increased Traffic, Handle More Request

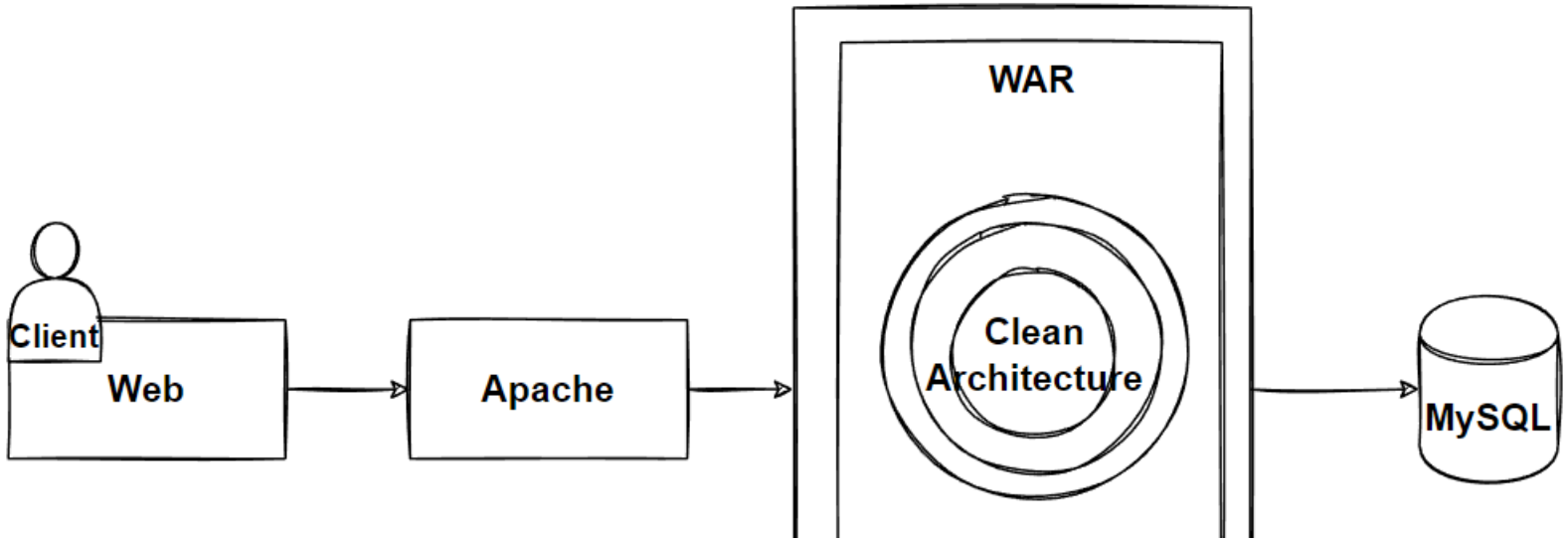


Problems

- Our E-Commerce Business is growing
- Need to handle greater amount of request per second
- Provide acceptable latency for users

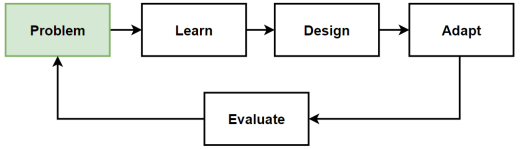
Solutions

- Scalabilitiy
- Vertical and Horizontal Scaling
- Scale Up and Scale Out
- Load Balancer



Concurrent Users	Requests/second	Latency (Expected)
2K	0.5K	
20K	12K	
100K	80K	<= 2 sec
500K	300K	?

Problem: Break Down Application into Microservices

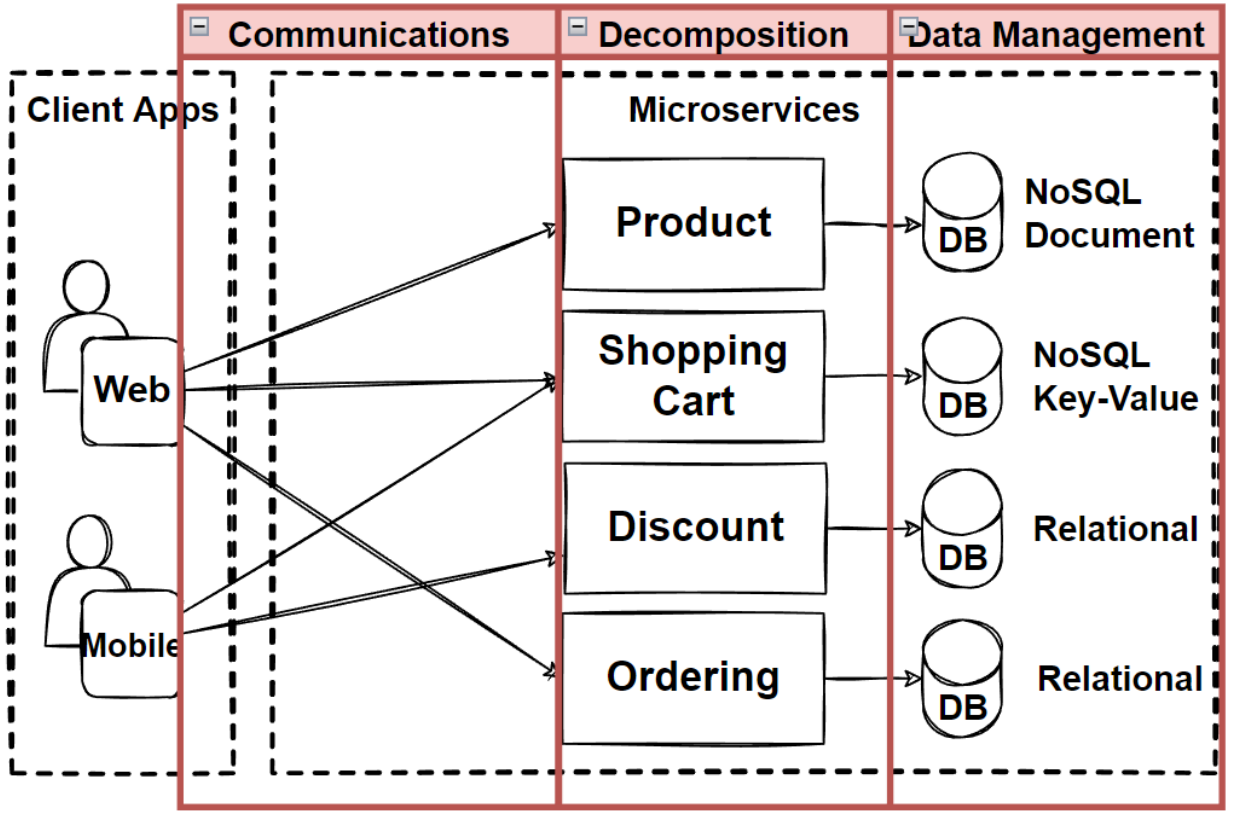


Problems

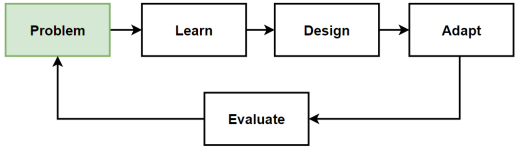
- Our E-Commerce Business is growing
- Teams wants to agile and add new features immediately to compete the market
- Required Independent Scale and Deployments
- We should clearly identify microservices which parts could be independent scale and deploy

Solutions

- Microservices Decomposition Patterns



Problem: Direct Client-to-Service Communication

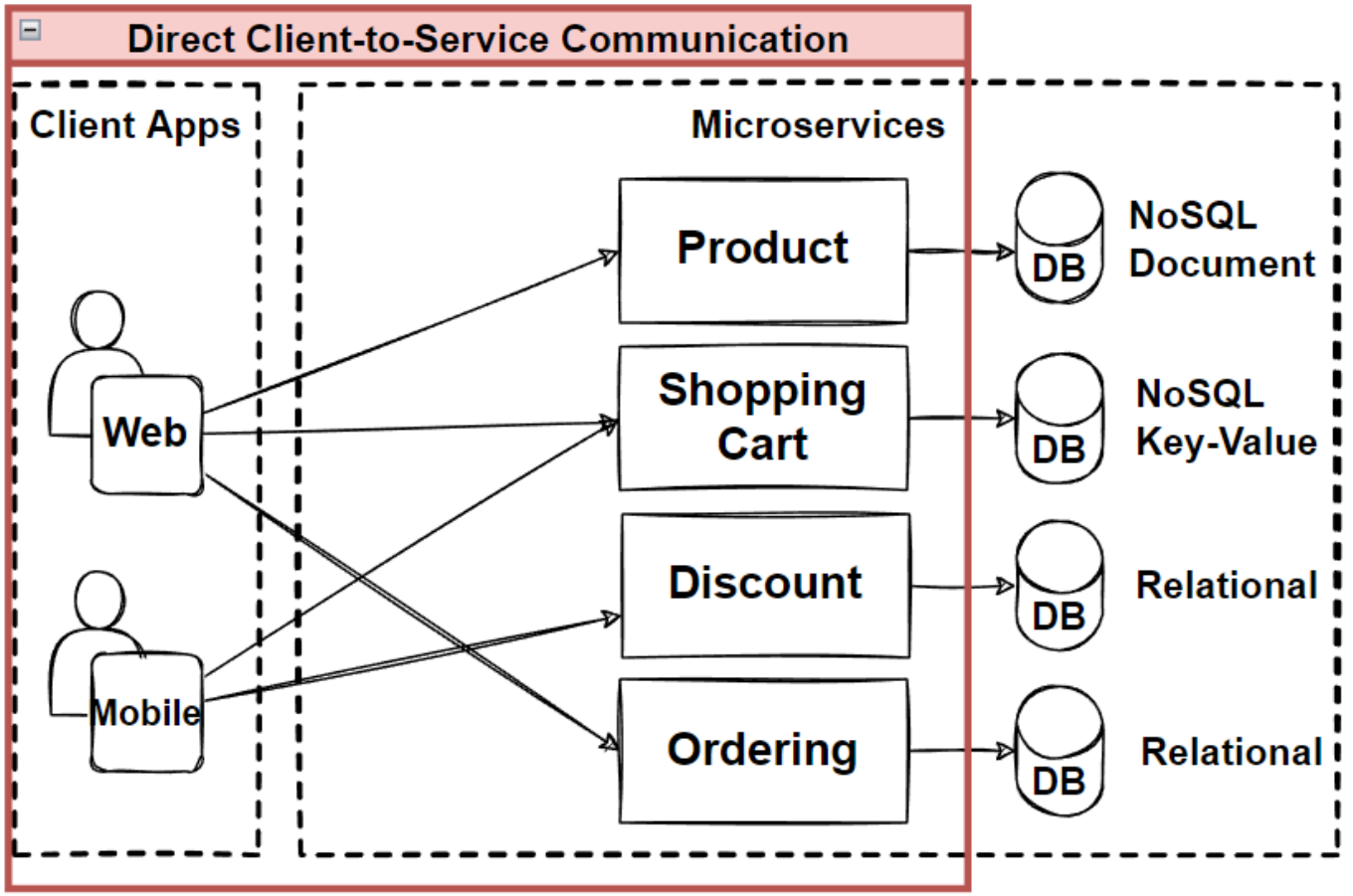


Problems

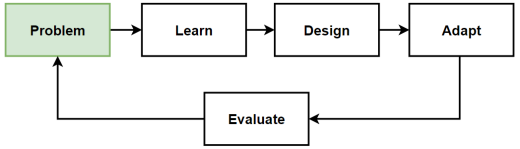
- Direct Client-to-Service Communication
- Cause to chatty calls from client to service
- Hard to manage invocations from client app.

Solutions

- Well-defined API Design
- Microservices Communication Patterns



Problem: Inter-service communication makes heavy load on network traffic



Problems

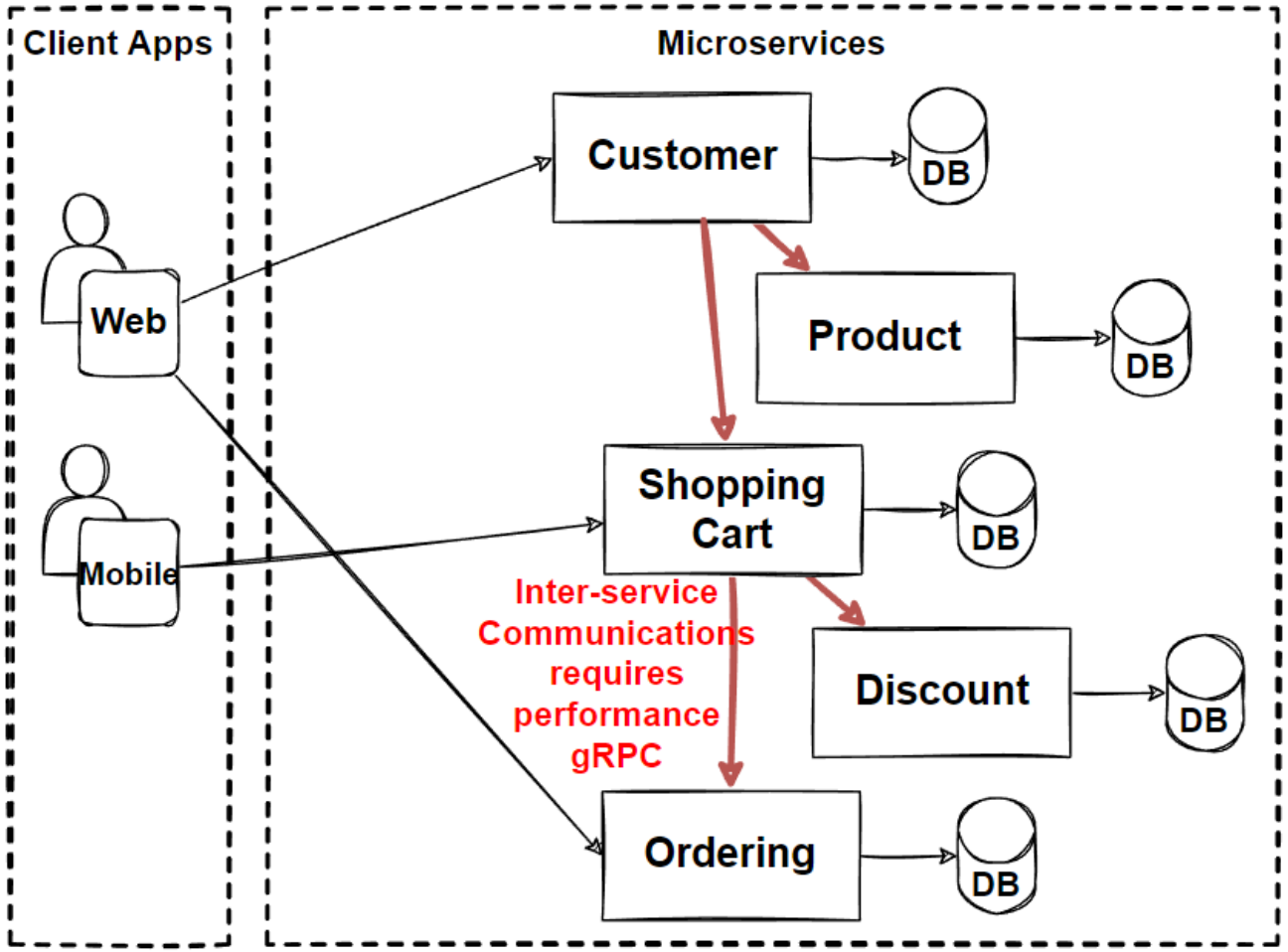
- Network performance issues on inter-service communication
- Backend Communication performance requirements
- Real-time communication requirements
- Streaming requirements

Example Use Case

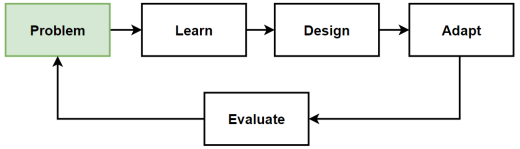
- Add Item into Shopping Cart that need to calculate with up-to-date discounts

Solutions

- gRPC APIs scalable and fast APIs
- Able to develop with different technologies with RPC framework



Problem: Chat with Support Agent



Problems

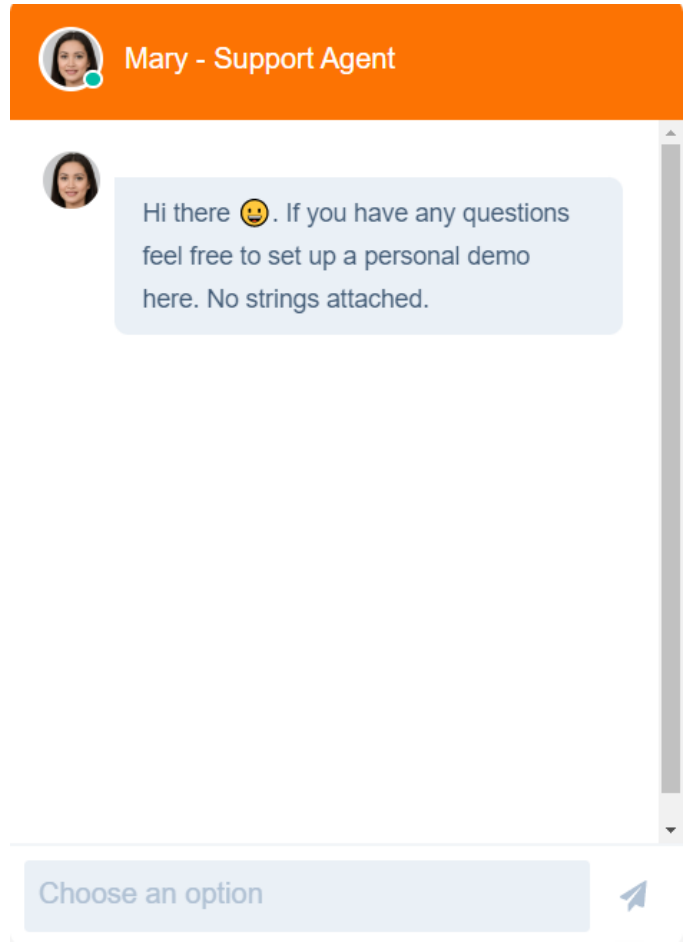
- Business teams request to answer Customer queries by chatting with Support Agents
- Real-time communication requirements
- Sending/receiving messages in Chat window

Example Use Case

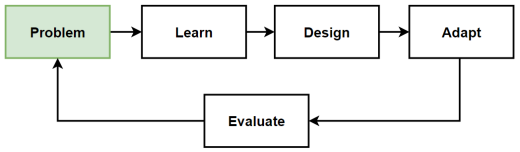
- E-commerce Online Agent help customer preferences as per product features on website

Solutions

- WebSocket APIs: Build real-time two-way communication applications



Problem: Service-to-Service Communications Chain Queries

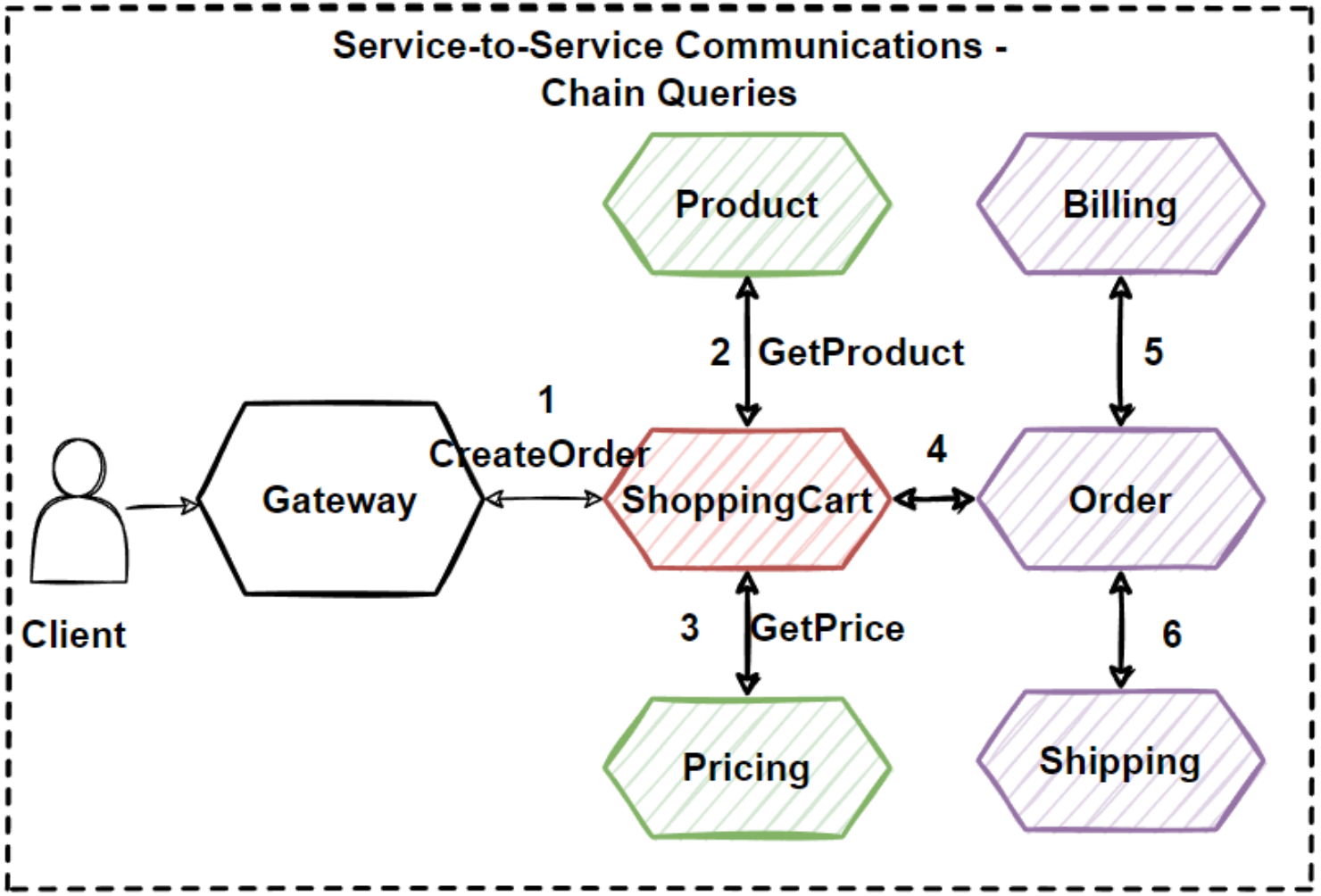


Problems

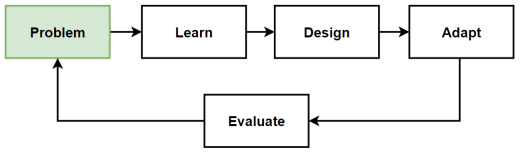
- HTTP calls to multiple microservices
- Chain Queries
- Visit more than a few microservices
- Increased latency

Solutions

- Aggregate query operations
- **Service Aggregator Pattern**



Problem: Long Running Operations Can't Handle with Sync Communication



Problems

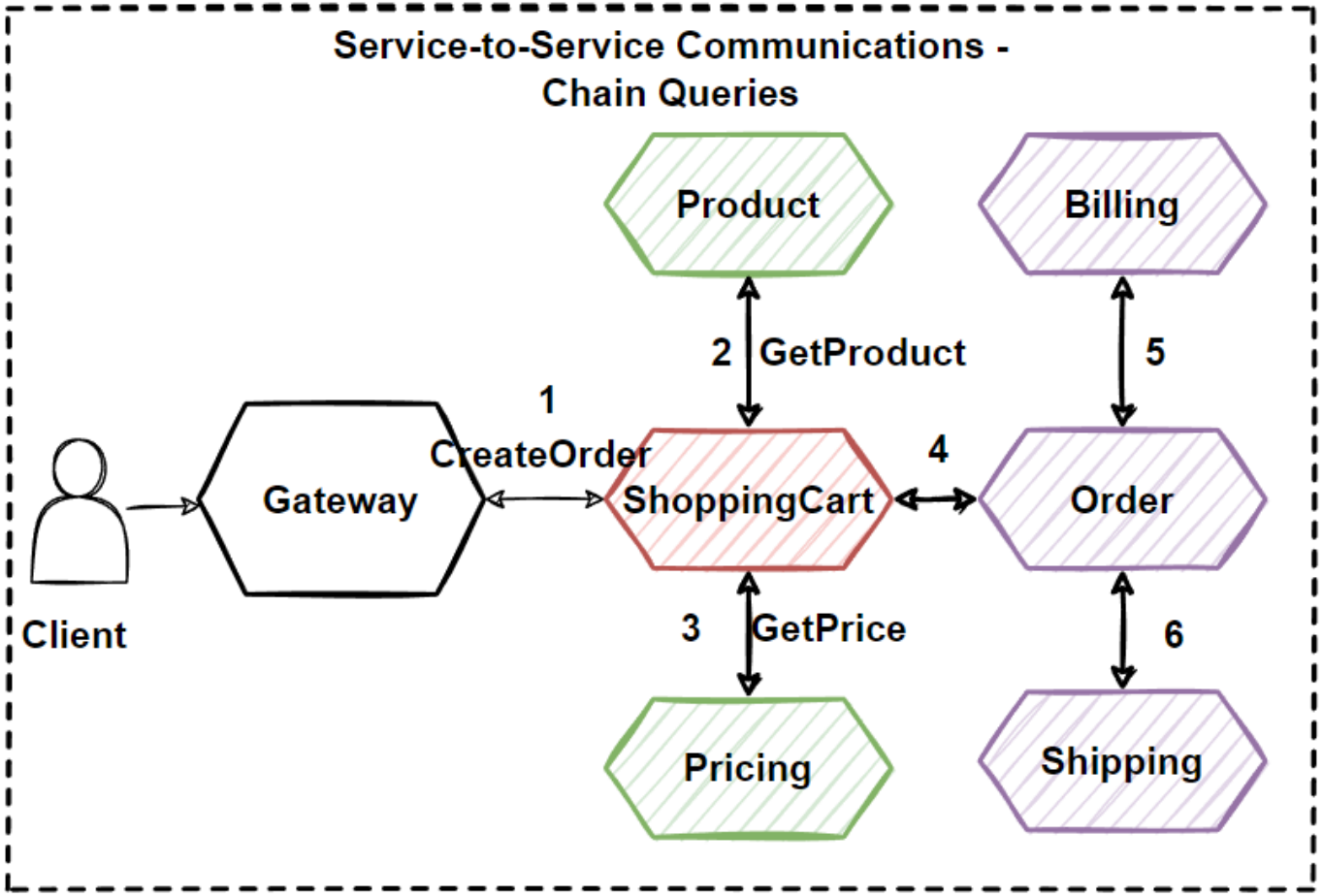
- HTTP calls to multiple microservices
- Chain Queries
- Visit more than a few microservices
- Increased latency with Highly Coupling Services
- Performance, scalability, and availability problems

Best Practices

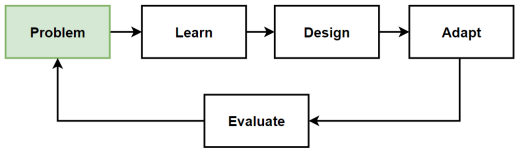
- Minimize the communication between the internal microservices
- make microservices communication in Asynchronous way as soon as possible.

Solutions

- **Asynchronous Message-Based Communications**
- Working with events



Problem: Database Bottlenecks when Scaling, Different Data Requirements For Microservices

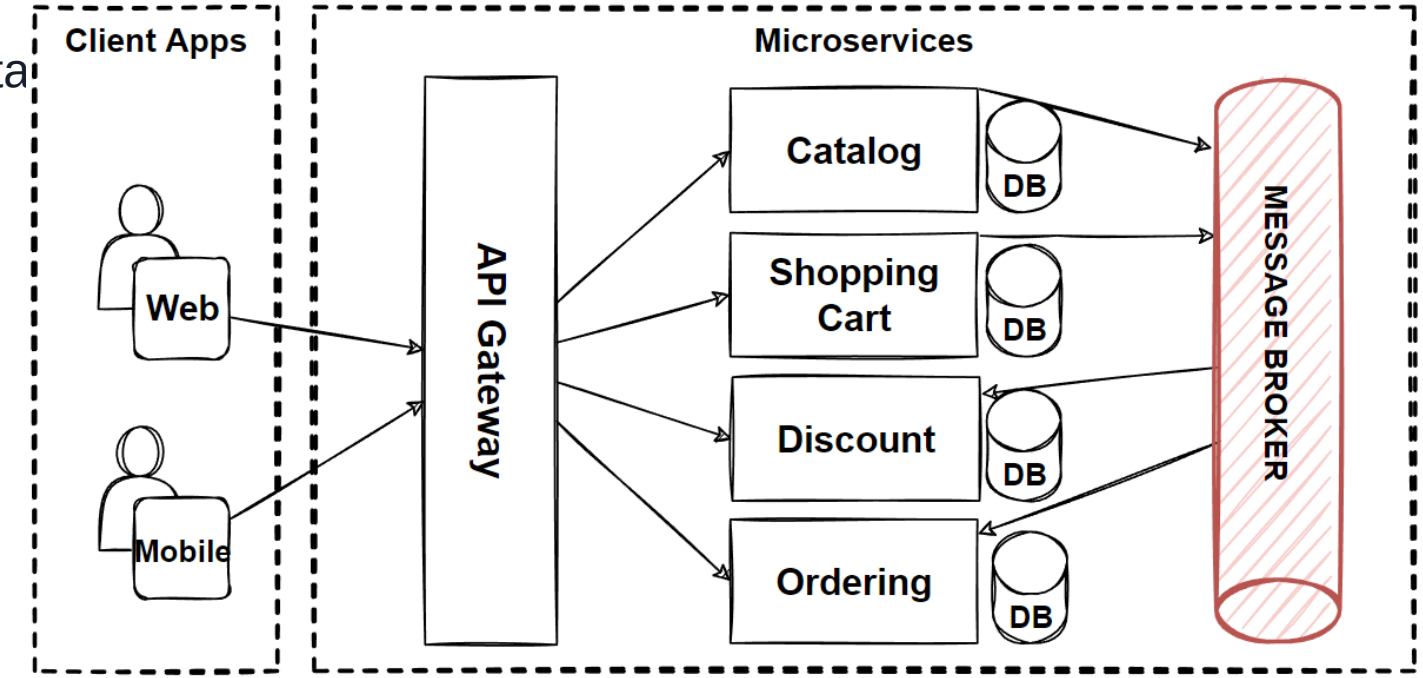


Problems

- Database are stateful service
- Scaling stateful services are not easy
- Vertical scaling has limits need to scale Horizonta
- Different Data Requirements For Microservices

Solutions

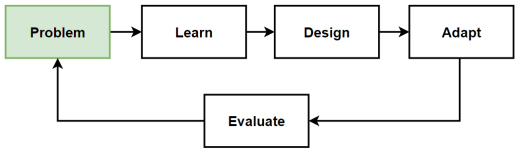
- Scale Stateful Application Horizontal Scaling
- Service and Data Partitioning along Business Boundaries - Shards/Pods
- Use NoSQL Database to gain partitioning
- Identify Database Requirements following best practices



Question

- **How to Choose a Database for Microservices ?**

Problem: Cross-Service Queries and Write Commands on Distributed Scaled Databases

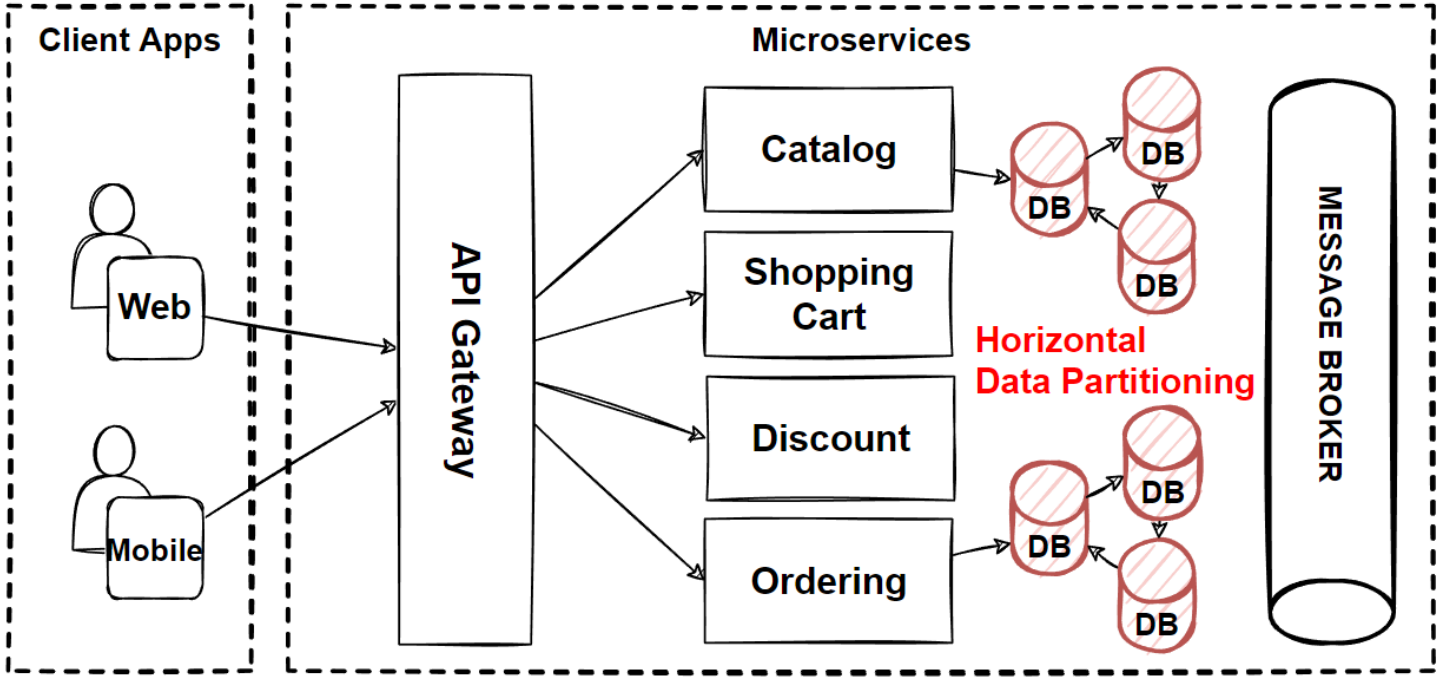


Considerations

- Cross-services queries that retrieve data from several microservices ?
- Separate read and write operations at scale ?

Problems

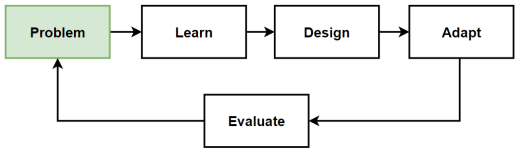
- Cross-Service Queries with Complex JOIN operations
- Read and write operations at scale
- Distributed Transaction Management



Solutions

- Microservices Data Query Pattern and Best Practices
- Materialized View Pattern
- CQRS Design Pattern
- Event Sourcing Pattern

Problem: Manage Consistency Across Microservices in Distributed Transactions



Considerations

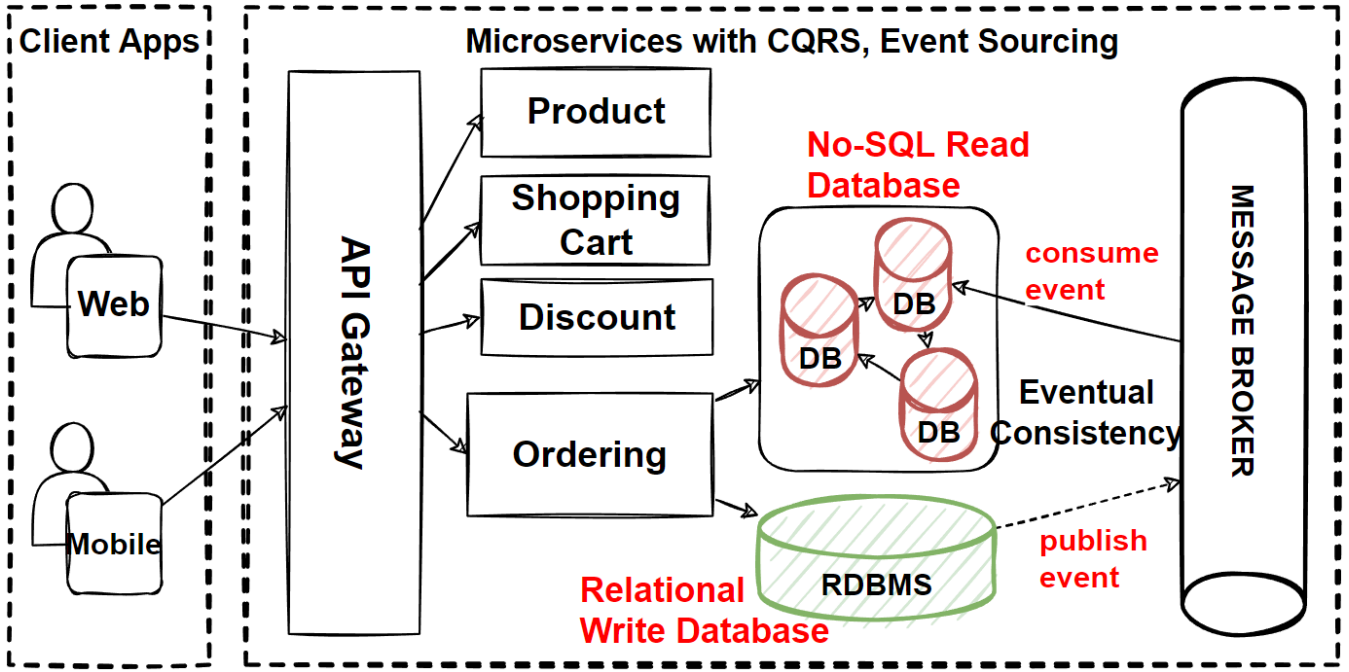
- Distributed Transactions that required to visit several microservices ?
- Consistency across multiple microservices ?
- Rollback transaction and run compensating steps ?

Problems

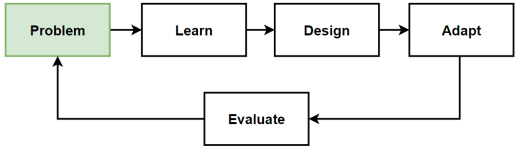
- Distributed Transaction Management
- Rollback Transaction on Distributed Environment
- Run Compensate Steps if one of service fail

Solutions

- Microservices Distributed Transaction Management Pattern and Best Practices
- Saga Pattern for Distributed Transactions
- Transactional Outbox Pattern
- Compensating Transaction pattern
- CDC - Change Data Capture



Problem: Handle Millions of Events Across Microservices

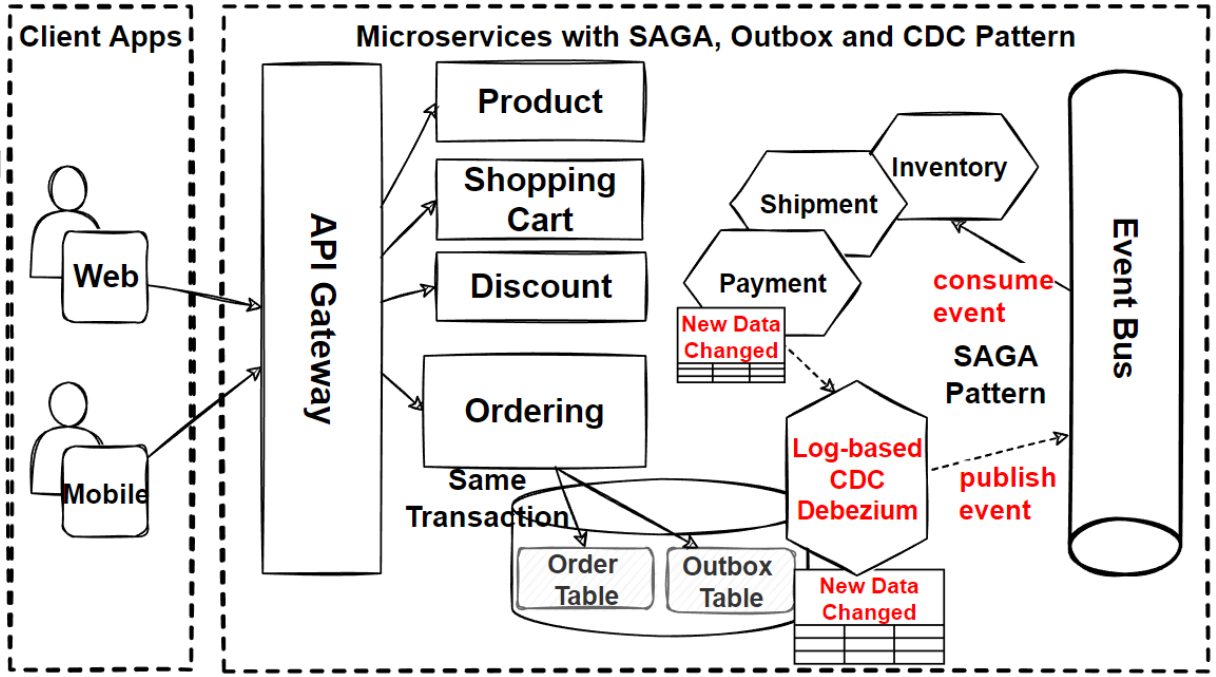


Considerations

- What if we have thousands of microservices that need to communicates with millions of events ?
- If multiple subsystems must process the same events
- Required Real-time processing with minimum latency.
- Required complex event processing, like pattern matching
- Required process high volume and high velocity of data, i.e. IoT apps.

Problems

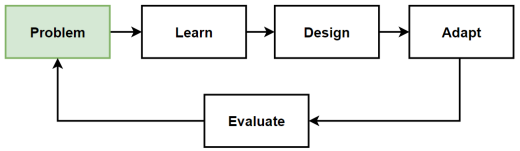
- Decoupled communications for thousands of microservices
- Real-time processing
- Handle High volume events



Solutions

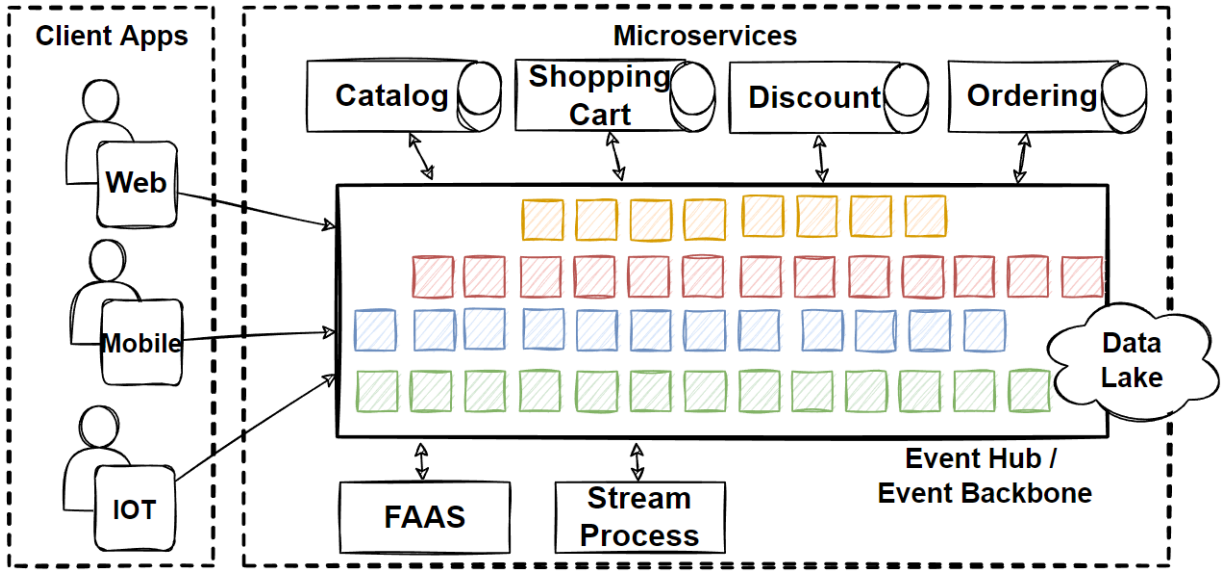
- Event-driven architecture for microservices

Problem: Database operations are expensive, low performance



Considerations

- Event-driven architecture comes with latency when publishing and subscribing events from the Event Hub.
- Sync REST APIs communication make expensive calls to a database that reduce performance.
- How can we make more faster that increase performance of communications in Microservices Architecture ?



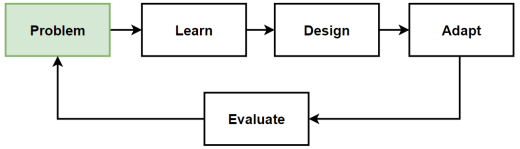
Problems

- Slowliness and Low Performance Communication
- Latency when publishing and subscribing events
- Rest APIs make Database calls that are expensive, low performance

Solutions

- Distributed cache
- Storing frequently accessed data in a distributed cache

Problem: Deploy Microservices at Anytime with Zero-downtime and flexible scale

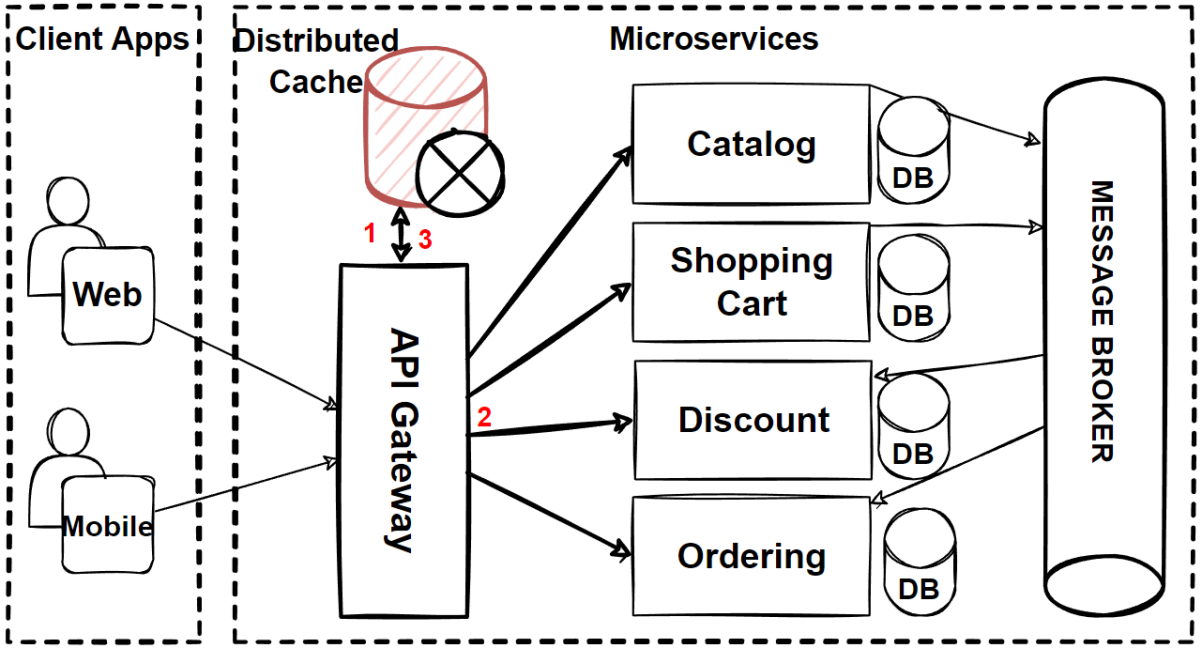


Problems

- Business teams wants to add new features immediately
- Innovate and experiment with new features
- Deploy features immediately, not waiting for deployment dates.
- Flexible scale for market peek times

Considerations

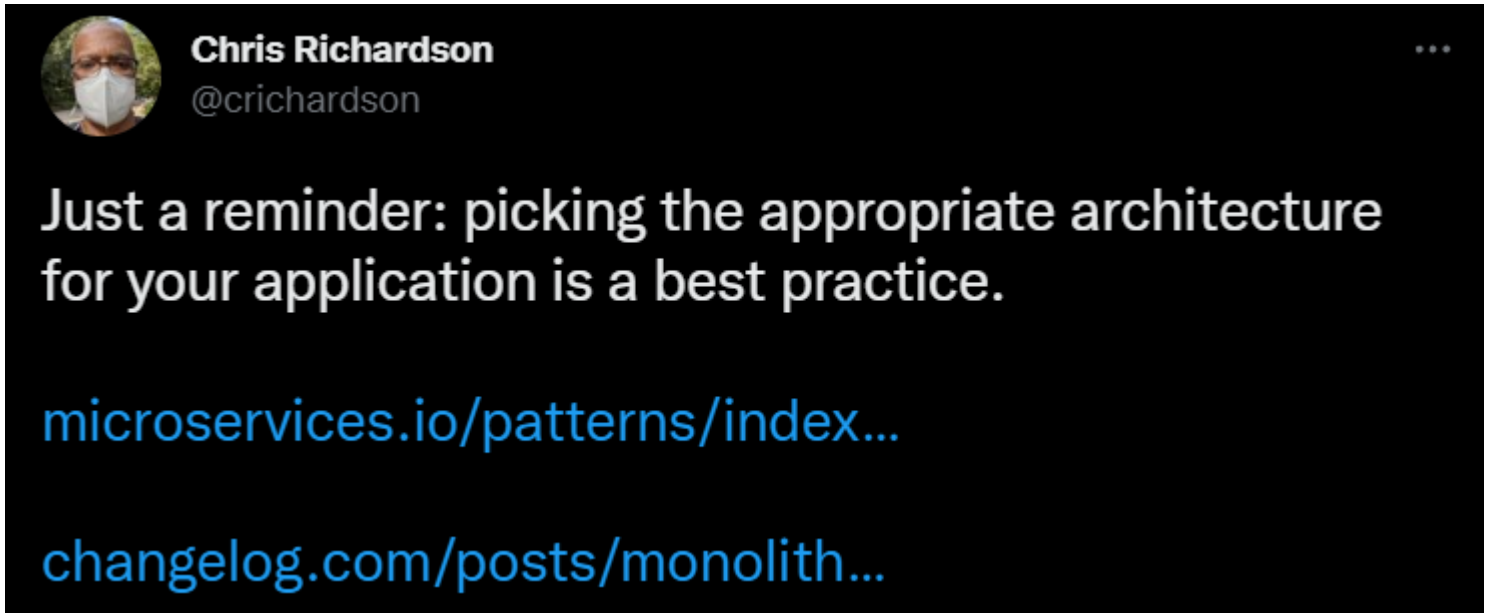
- Ensure continuity of service and minimize disruption
- Allow for continuous delivery
- Support high-traffic environments

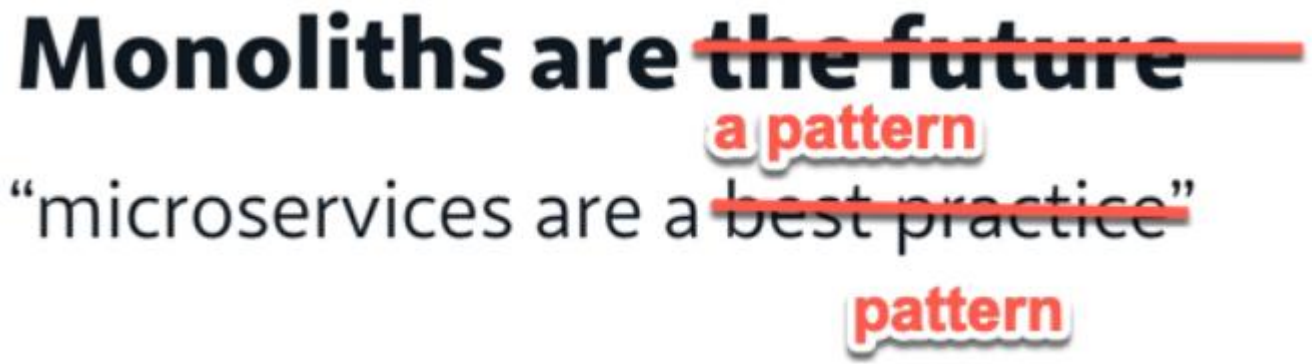


Solutions

- Containers and Orchestrators
- Deployment strategies; blue-green deployment, rolling deployment, and canary deployment.
- Kubernetes Patterns; Sidecar Patterns, Service Mesh Pattern
- DevOps and CI/CD Pipelines and Infrastructure as code (IaC)

Choosing the Right Architecture for your Application



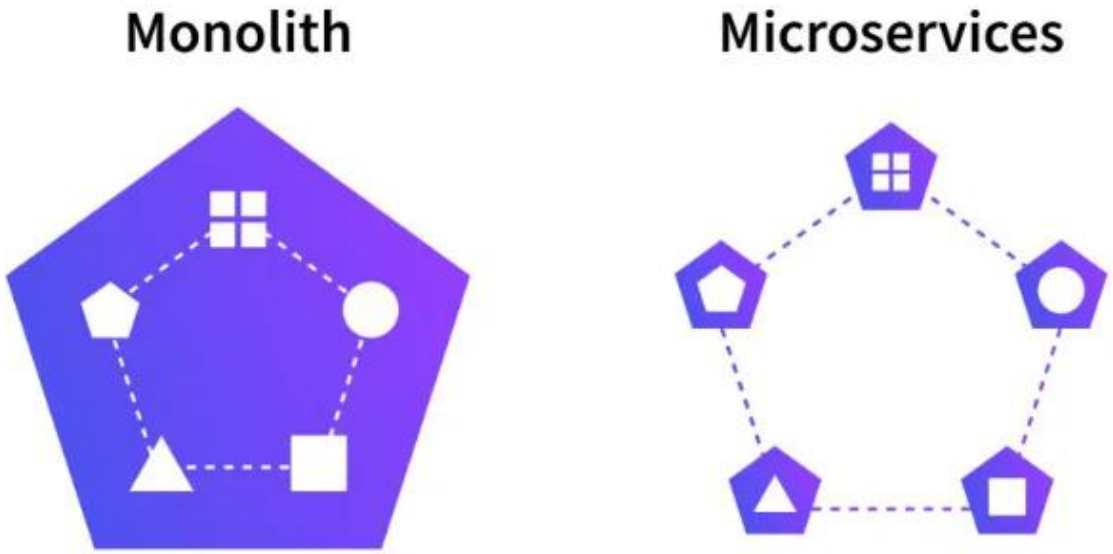


Choosing the Right Architecture for your Application

- The Majestik Monolithic

Every Architecture has Pros and Cons

- Understand that **every architecture** has **benefits** and also **drawbacks**
- Consider every architecture style as a SA
- What is the Right Architecture for your Application?
- **It Depends**..
- Monolithic Architecture and Microservice Architecture are **architectural patterns**.
- No architecture pattern **is better than other.**
- Design your system with focusing on **context** and **non-functional requirements (-ilities)**
- **WRONG:** Microservices > Monolithic



Design for Business Requirements

- Every **design decision** must be **justified** by **a business requirement.**
- T**o avoid over-engineering** the application architectures, keep to drive design for business requirement.
- Engineers are **tend to be over-engineering** with latest architecture styles, use latest tools.
- It **becomes** an **experimental application** that use all latest fancy tools and architectures.
- We should clearly **define functional** and **nonfunctional** requirements.
- **Define** our **limits**, **constraints** and **assumptions** of the application and define business objectives clearly.
- **Start** and **Grow Application** with **Metrics**
  - How many Concurrent Users that our application handle ?
  - What is the target of expected Requests/second Latency ?
  - What level of application outage is acceptable?

- **Business requirements drive** these **design considerations.**

Way of Learning – The Course Flow